WEB ATTACK CHAIN
ANALYSIS REPORT

Threat Detection & Incident Documentation

Prepared by: Security Operations Team

Detection Platform: Elastic SIEM

Date: March 2026

Classification: CONFIDENTIAL

EXECUTIVE SUMMARY

This report documents a complete web attack chain detected and investigated through Elastic SIEM on a target environment running Apache web services. A total of eleven (11) distinct attack techniques were identified, spanning the full lifecycle of a sophisticated intrusion — from initial reconnaissance and brute force credential attacks through to post-exploitation, lateral movement, and data exfiltration attempts.

All detections were correlated from Apache access logs, network packet captures (Wireshark/PCAP), and Elastic SIEM alert rules. Each attack has been individually documented with supporting evidence, timestamps, network indicators, and recommended remediation actions.

Summary of Detected Attacks

#	Attack Name	Severity	Status
01	Investigating Login Brute Forcing Activity	High	Documented
02	Analyzing Subdomain Enumeration Activity	High	Documented
03	Investigating SQL Injection Union Attack	Critical	Documented
04	Investigating SQLMap Attack Detection	Critical	Documented
05	Investigating Directory Traversal Attack	Critical	Documented
06	Investigating Classic SQL Injection Detection	Critical	Documented
07	Investigating IDOR Attack	High	Documented
08	Investigating File Upload Vulnerability Attack	Critical	Documented
09	Investigating XSS Vulnerability Attack	High	Documented
10	Investigating SSRF	Critical	Documented
11	Investigating Command Injection	Critical	Documented

ATTACK 01

Investigating Login Brute Forcing Activity

Attack Overview

Attack ID	SO-WEB-01
Attack Type	Brute Force / Credential Attack
Severity	High
Detection Source	Elastic SIEM — Apache Access Logs
Target Host	btweb-virtual-machine
Target IP	172.16.25.15
Source IP	10.20.10.3
Target Endpoint	/login.php
Protocol	HTTP/1.1
Destination Port	80
HTTP Method	POST
Response Code	404 (Failure)
Timestamp (First)	Apr 25, 2025 @ 15:24:19.000
Timestamp (Last)	Apr 25, 2025 @ 15:24:24.000+
Total Events	52 matching events detected
Agent Type	Filebeat 8.18.0
ECS Version	1.12.0

Attack Description

A Login Brute Force attack is a systematic, automated technique in which an adversary repeatedly submits credentials — typically via POST requests to a login endpoint — in an attempt to gain unauthorized access to an application or system. The attacker cycles through large lists of username/password combinations, relying on volume and automation to eventually identify valid credentials.

In this incident, the attacker at IP 10.20.10.3 systematically targeted the /login.php endpoint on the victim server at 172.16.25.15. The attack manifested as a high-frequency sequence of HTTP POST requests using application/x-www-form-urlencoded encoding — the standard content type for web form submissions — all resulting in 404 failure responses from the server.

The events were captured across two detection surfaces: Elastic SIEM ingesting Apache access logs via Filebeat, and network-level packet capture in Wireshark showing raw TCP traffic between the attacker and the web server on port 80.

Technical Analysis

Key indicators observed across both detection sources are detailed below:

Indicator	Observed Value / Detail
Query Filter Used	event.dataset: apache.access AND url.path.keyword: *login* AND event.outcome: failure AND http.response.status_code: 404 AND http.request.method: POST
Packet Filter (Wireshark)	http.request.uri == "/login.php" and http.request.method == "POST"
Content-Type Header	application/x-www-form-urlencoded
Packet Frame (First)	Frame 1471 — 628 bytes on wire
Source MAC	VMware_96:ef:ce (00:50:56:96:ef:ce)
Destination MAC	VMware_76:66:76 (00:0c:29:76:66:76)
TCP Source Port	52872
TCP Destination Port	80 (HTTP)
Request Interval	~1 second between each POST request
Total Requests (PCAP)	15+ visible back-to-back requests in capture
Event Category	web
Agent Name	btweb-virtual-machine

Attack Timeline

Timestamp	Event
Apr 25, 2025 @ 15:24:19.000	First brute force POST request detected to /login.php — event outcome: failure
Apr 25, 2025 @ 15:24:20.000	Second consecutive POST failure — automated pattern begins
Apr 25, 2025 @ 15:24:21.000	Third POST request — 1-second interval confirms automated tooling
Apr 25, 2025 @ 15:24:22.000	Fourth POST failure recorded — all from same agent ID
Apr 25, 2025 @ 15:24:23.000	Fifth POST failure — 52 total matching events ultimately ingested
Apr 25, 2025 @ 15:24:24.000	Sixth+ sequential failure — Wireshark PCAP shows 15+ raw frames captured
Apr 25, 2025 @ 15:28:35.592	Events ingested into Elastic (event.created timestamp)
Apr 25, 2025 @ 15:28:45.649	Final event ingestion completed (event.ingested timestamp)

Evidence & Screenshots

Figure 1 — Elastic SIEM: Apache Access Log Query Results

The Elastic SIEM query below confirms 52 total events matching the brute force pattern. Each event shows event.dataset: apache.access, event.outcome: failure, http.request.method: POST, captured via Filebeat agent on the target host btweb-virtual-machine.

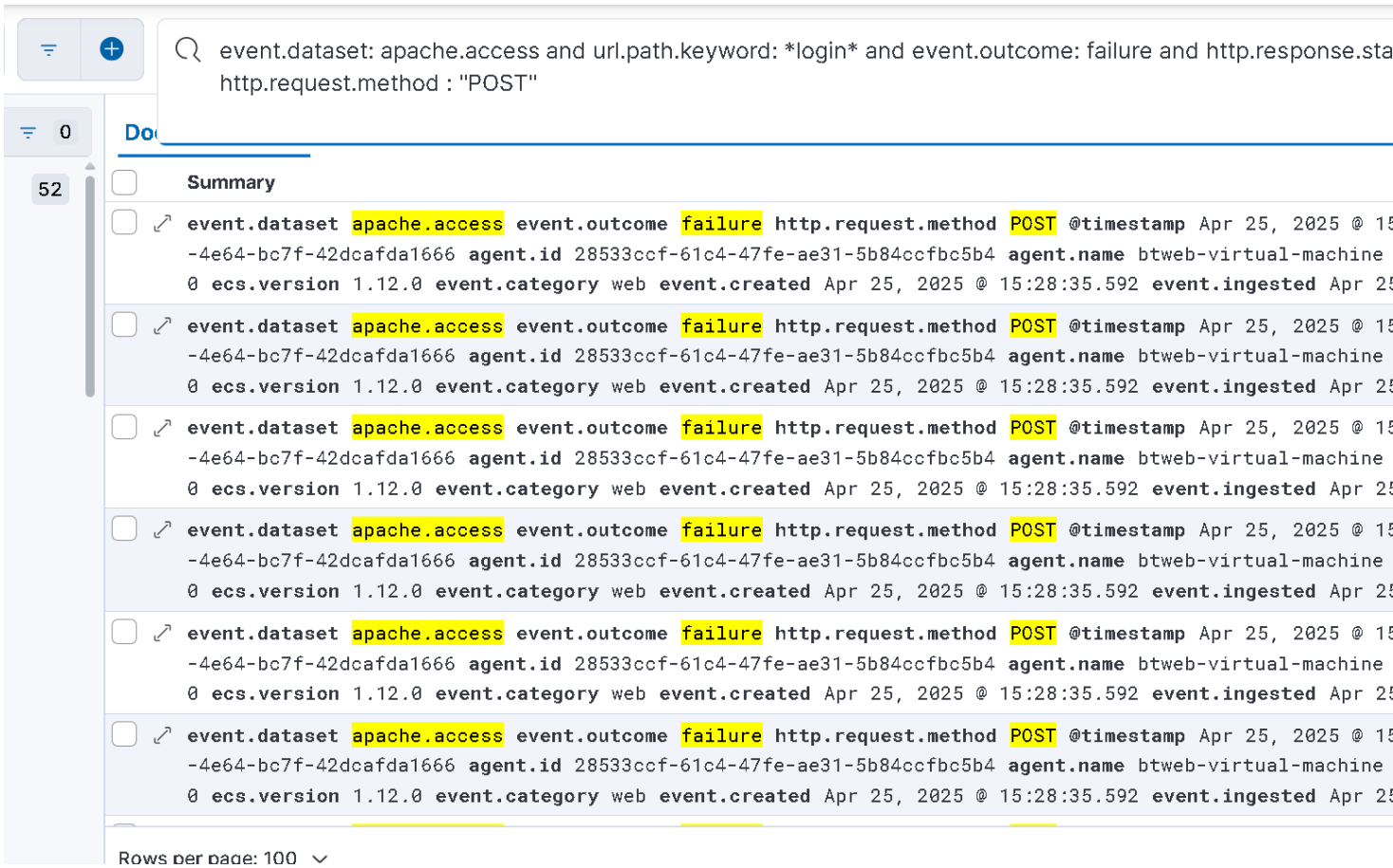
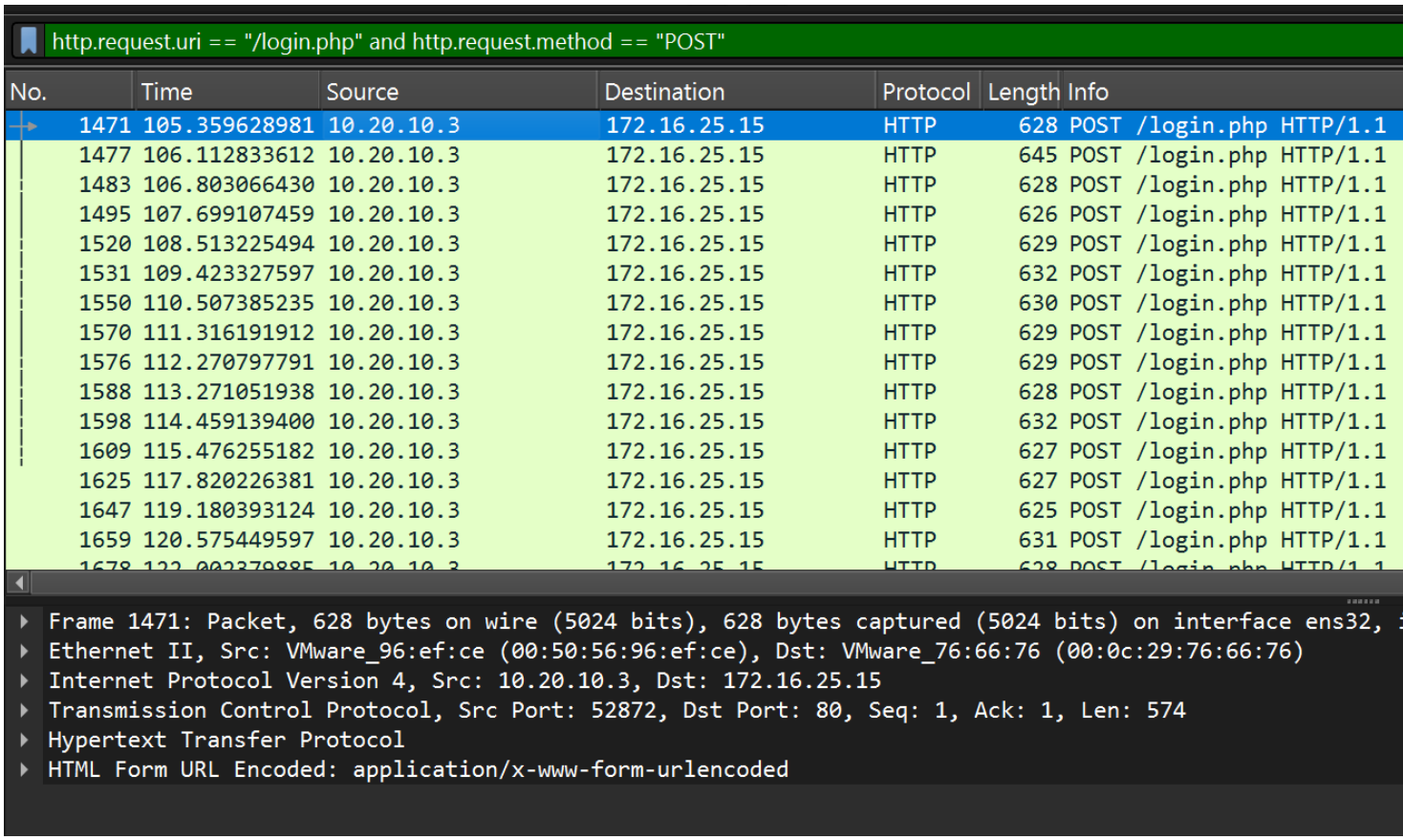


Figure 1: Elastic SIEM query showing 52 brute force login failure events from 10.20.10.3

Figure 2 — Wireshark PCAP: Network-Level Packet Capture

The Wireshark packet capture corroborates the Elastic findings at the network level. The filter http.request.uri == "/login.php" and http.request.method == "POST" reveals a sustained sequence of POST requests originating from 10.20.10.3 targeting 172.16.25.15 on TCP port 80. Packet frame details confirm application/x-www-form-urlencoded form data transmission.



http.request.uri == "/login.php" and http.request.method == "POST"

No.	Time	Source	Destination	Protocol	Length	Info
1471	105.359628981	10.20.10.3	172.16.25.15	HTTP	628	POST /login.php HTTP/1.1
1477	106.112833612	10.20.10.3	172.16.25.15	HTTP	645	POST /login.php HTTP/1.1
1483	106.803066430	10.20.10.3	172.16.25.15	HTTP	628	POST /login.php HTTP/1.1
1495	107.699107459	10.20.10.3	172.16.25.15	HTTP	626	POST /login.php HTTP/1.1
1520	108.513225494	10.20.10.3	172.16.25.15	HTTP	629	POST /login.php HTTP/1.1
1531	109.423327597	10.20.10.3	172.16.25.15	HTTP	632	POST /login.php HTTP/1.1
1550	110.507385235	10.20.10.3	172.16.25.15	HTTP	630	POST /login.php HTTP/1.1
1570	111.316191912	10.20.10.3	172.16.25.15	HTTP	629	POST /login.php HTTP/1.1
1576	112.270797791	10.20.10.3	172.16.25.15	HTTP	629	POST /login.php HTTP/1.1
1588	113.271051938	10.20.10.3	172.16.25.15	HTTP	628	POST /login.php HTTP/1.1
1598	114.459139400	10.20.10.3	172.16.25.15	HTTP	632	POST /login.php HTTP/1.1
1609	115.476255182	10.20.10.3	172.16.25.15	HTTP	627	POST /login.php HTTP/1.1
1625	117.820226381	10.20.10.3	172.16.25.15	HTTP	627	POST /login.php HTTP/1.1
1647	119.180393124	10.20.10.3	172.16.25.15	HTTP	625	POST /login.php HTTP/1.1
1659	120.575449597	10.20.10.3	172.16.25.15	HTTP	631	POST /login.php HTTP/1.1

```
Frame 1471: Packet, 628 bytes on wire (5024 bits), 628 bytes captured (5024 bits) on interface ens32,
Ethernet II, Src: VMware_96:ef:ce (00:50:56:96:ef:ce), Dst: VMware_76:66:76 (00:0c:29:76:66:76)
Internet Protocol Version 4, Src: 10.20.10.3, Dst: 172.16.25.15
Transmission Control Protocol, Src Port: 52872, Dst Port: 80, Seq: 1, Ack: 1, Len: 574
Hypertext Transfer Protocol
HTML Form URL Encoded: application/x-www-form-urlencoded
```

Figure 2: Wireshark PCAP showing sequential POST /login.php requests from 10.20.10.3 to 172.16.25.15:80

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Credential Access	**T1110**	Brute Force
Credential Access	**T1110.001**	Password Guessing
Initial Access	**T1190**	Exploit Public-Facing Application

Recommendations & Remediation

The following controls and measures are recommended to prevent and mitigate login brute force attacks:

- Implement account lockout policies after a defined number of consecutive failed login attempts (e.g., 5 attempts within 60 seconds).
- Deploy rate limiting and throttling on the /login.php endpoint to restrict the number of POST requests per IP per time window.
- Enable CAPTCHA or multi-factor authentication (MFA) on all web-facing login forms to defeat automated tooling.

- Block or geo-restrict the source IP 10.20.10.3 at the firewall or WAF level pending investigation.
- Review and harden the Apache web server configuration to return appropriate error codes and avoid information leakage.
- Deploy a Web Application Firewall (WAF) with brute force detection signatures enabled.
- Establish Elastic SIEM alerting rules to trigger on more than 10 failed POST requests to login endpoints within 60 seconds from a single source IP.
- Regularly audit authentication logs and review anomalous patterns of failed logins during off-hours.

ATTACK 02

Analyzing Subdomain Enumeration Activity

Attack Overview

Attack ID	SO-WEB-02
Attack Type	Reconnaissance / Subdomain & Directory Enumeration
Severity	High
Detection Source	Elastic SIEM — Apache Access Logs
Target Host	btweb-virtual-machine
Tool(s) Identified	Wfuzz 3.1.0, massdns, nikto, dirb
HTTP Response Code	404 (Failure — resources not found)
Event Outcome	failure
Timestamp (First)	Apr 28, 2025 @ 12:01:02.000
Timestamp (Second Burst)	Apr 28, 2025 @ 12:01:48.000
Event Created	Apr 28, 2025 @ 12:01:23.362 / 12:01:51.298
Event Ingested	Apr 28, 2025 @ 12:09:39.378 – 12:09:39.860
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0
ECS Version	1.12.0
Event Category	web

Attack Description

Subdomain and web directory enumeration is a reconnaissance technique used by attackers to map the attack surface of a target web application or server. By systematically probing for existing subdomains, hidden directories, files, and endpoints, the attacker seeks to identify entry points that are not publicly advertised but may be accessible — such as admin panels, backup files, configuration pages, or legacy application paths.

In this incident, the attacker employed well-known enumeration tools — most prominently Wfuzz 3.1.0 — to conduct automated fuzzing of the target web server. The Elastic SIEM query detected events matching the known User-Agent strings of Wfuzz/3.1.0, massdns, nikto, and dirb in the Apache access logs. All requests resulted in HTTP 404 failure responses, confirming the enumeration nature of the activity.

Wfuzz is a web application fuzzer designed to brute-force web content such as directories, files, and parameters. Nikto is an open-source web server scanner checking for dangerous files and

common vulnerabilities. Dirb is a web content scanner using dictionary-based attacks. Massdns is a high-performance DNS resolver used for bulk subdomain resolution. The combination of these tools indicates a thorough, structured reconnaissance campaign.

Technical Analysis

Key indicators extracted from Elastic SIEM and Apache access logs:

Indicator	Observed Value / Detail
Elastic Query Filter	event.dataset: apache.access AND event.outcome: failure AND http.response.status_code: 404 AND user_agent.original: ("Wfuzz/3.1.0" or "massdns" or "nikto" or "dirb")
Primary Tool Detected	Wfuzz/3.1.0 (User-Agent string confirmed in all captured events)
Additional Tools in Filter	massdns, nikto, dirb
HTTP Response Code	404 — target paths/subdomains not found
Event Outcome	failure — no successful resource access
Attack Pattern	Automated high-frequency HTTP requests with enumeration tool User-Agents
First Event Timestamp	Apr 28, 2025 @ 12:01:02.000
Second Burst	Apr 28, 2025 @ 12:01:48.000 (multiple events at same second)
Ingestion Delay	~8 minutes between event occurrence and Elastic ingestion
Agent	Filebeat 8.18.0 on btweb-virtual-machine

Enumeration Tools Identified

Tool	Purpose	Detection Method
Wfuzz 3.1.0	Web content fuzzer — directory & parameter brute-force	User-Agent: Wfuzz/3.1.0 in Apache access logs
massdns	High-speed DNS subdomain resolver	User-Agent: massdns in Apache access logs
nikto	Web server vulnerability scanner	User-Agent: nikto in Apache access logs
dirb	Web content dictionary scanner	User-Agent: dirb in Apache access logs

Attack Timeline

Timestamp	Event
Apr 28, 2025 @ 12:01:02.000	First enumeration request detected — User-Agent: Wfuzz/3.1.0 — HTTP 404 failure
Apr 28, 2025 @ 12:01:23.362	Event created timestamp recorded by Filebeat on host

Apr 28, 2025 @ 12:01:48.000	Second burst — multiple concurrent events at same second confirming automated scanning
Apr 28, 2025 @ 12:01:51.298	Event created for second burst — 3 seconds post-request
Apr 28, 2025 @ 12:09:39.378	First event ingested into Elastic SIEM (~8 min pipeline delay)
Apr 28, 2025 @ 12:09:39.854 – .860	Rapid successive ingestion of all remaining enumeration events

Evidence & Screenshot

Figure 3 — Elastic SIEM: Wfuzz/3.1.0 Enumeration Activity in Apache Access Logs

The screenshot below shows the Elastic SIEM query results filtering Apache access logs for known enumeration tool User-Agent strings. All returned events display event.outcome: failure and HTTP 404 responses, with Wfuzz/3.1.0 clearly identified as the primary tool. Multiple events share identical timestamps (12:01:48.000), confirming automated high-frequency scanning behavior.

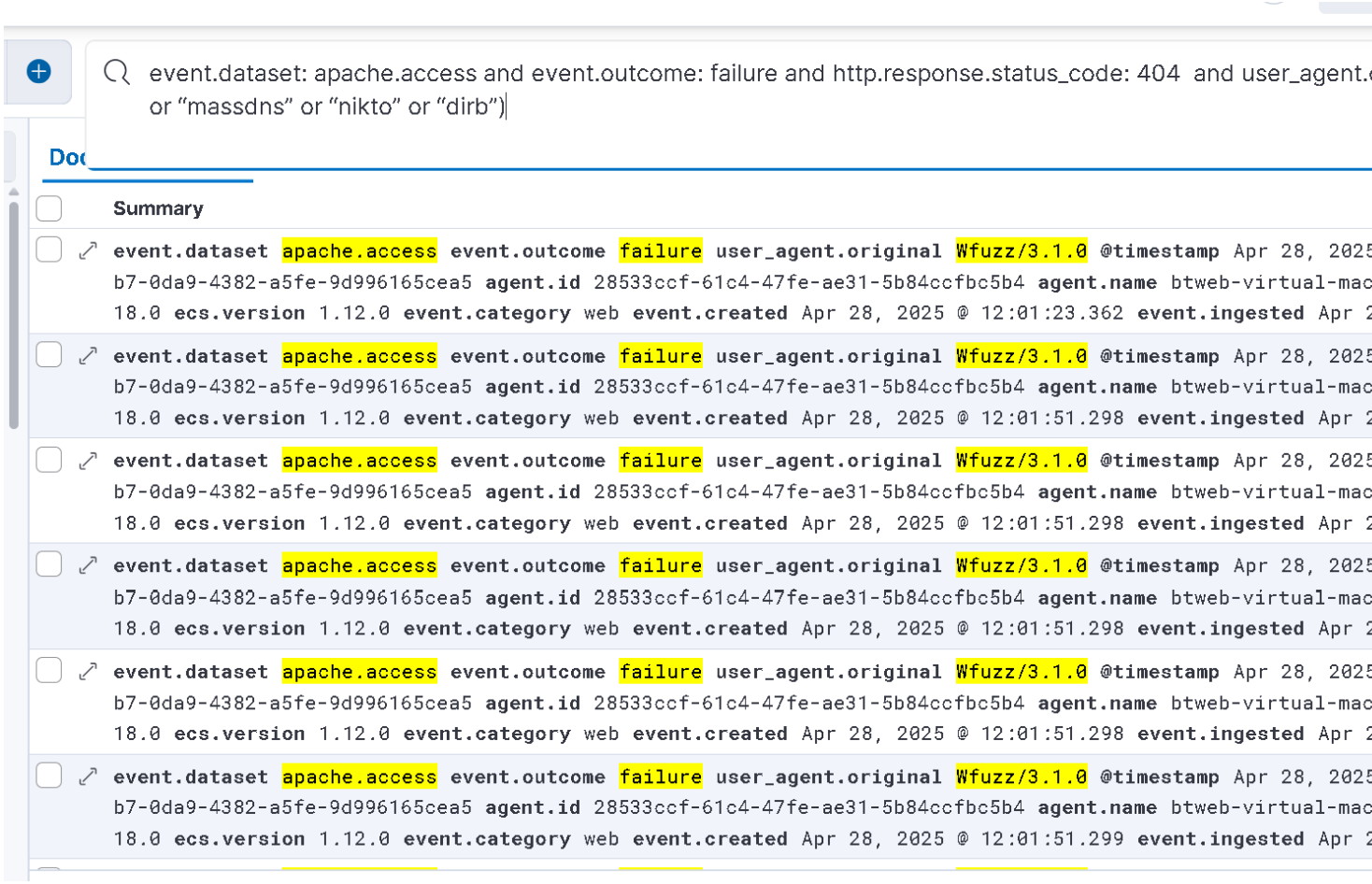


Figure 3: Elastic SIEM — Wfuzz/3.1.0 subdomain enumeration events with HTTP 404 failures on btweb-virtual-machine

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Reconnaissance	**T1595**	Active Scanning
Reconnaissance	**T1595.003**	Wordlist Scanning
Reconnaissance	**T1590**	Gather Victim Network Information
Discovery	**T1083**	File and Directory Discovery

Recommendations & Remediation

The following measures are recommended to detect and prevent web enumeration attacks:

- Block or rate-limit known enumeration tool User-Agent strings (Wfuzz, nikto, dirb, massdns) at the WAF or reverse proxy level.
- Implement Elastic SIEM detection rules to alert on more than 20 HTTP 404 responses from a single IP within any 60-second window.
- Deploy honeypot directories and files that trigger alerts when accessed, helping detect enumeration attempts early.
- Enable IP-based rate limiting on the web server (Apache mod_evasive or equivalent) to throttle high-frequency requestors.
- Regularly review and restrict directory listing and access to sensitive paths in the Apache configuration.
- Use Threat Intelligence feeds to proactively block IPs associated with scanning and enumeration activity.
- Ensure all non-public directories are protected with authentication and not accessible to unauthenticated users.

ATTACK 03

Investigating SQL Injection Union Attack

Attack Overview

Attack ID	SO-WEB-03
Attack Type	SQL Injection — UNION-Based Data Extraction
Severity	Critical
Detection Source	Elastic SIEM — Apache Access Logs
Target Host	btweb-virtual-machine
Target Endpoint	/database/
Injection Parameter	value
Injection User	alice_wonder (injected as base value)
HTTP Response Code	200 (Successful — server returned data)
HTTP Method	GET
Response Size	1091 bytes
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
Timestamp (First)	Apr 28, 2025 @ 16:53:29.000
Timestamp (Last)	Apr 28, 2025 @ 16:54:07.000
Total Events	5 distinct UNION SELECT payloads detected
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0
ECS Version	1.12.0
Event Category	web

Attack Description

A SQL Injection (SQLi) UNION attack is one of the most dangerous and directly impactful forms of SQL injection. In a UNION-based attack, the adversary appends a UNION SELECT statement to the original SQL query processed by the application, forcing the database to return data from additional tables alongside — or instead of — the legitimate query results. This technique allows the attacker to extract sensitive data from any table in the database, including credentials, user records, schema metadata, and application secrets.

In this incident, the attacker targeted the /database/ endpoint using the value GET parameter with the base value alice_wonder. By appending progressively sophisticated UNION SELECT payloads, the attacker first determined the number of columns in the original query using NULL-based probing

(UNION SELECT NULL,NULL,NULL and UNION SELECT NULL,NULL,NULL,NULL), then enumerated the database schema by querying information_schema.tables and information_schema.columns, ultimately targeting the login table to extract column names and potentially credentials.

Critically, all five injection requests returned HTTP 200 responses with a 1091-byte payload, confirming that the server processed and responded to the malicious queries successfully. The URL-encoded payloads were decoded using CyberChef, revealing the full SQL injection syntax. This is a high-confidence, confirmed data extraction event.

SQL Injection Payload Analysis

The following payloads were extracted from the url.query field and decoded from URL encoding:

#	Decoded Payload	Purpose / Stage
1	`value=alice_wonder' UNION SELECT NULL,NULL,NULL--`	Column count probing — testing for 3 columns in original query
2	`value=alice_wonder' UNION SELECT NULL,NULL,NULL,NULL--`	Column count probing — testing for 4 columns in original query
3	`value=alice_wonder' union select table_name,2,3 from information_schema.tables--`	Schema enumeration — listing all table names in the database
4	`value=alice_wonder' union select table_name,2,3 from information_schema.tables--`	Schema enumeration — repeated/confirmed table listing request
5	`value=alice_wonder' UNION SELECT column_name,2,3 FROM information_schema.columns WHERE table_name='login'--`	Targeted extraction — enumerating column names of the login table

Technical Analysis

Indicator	Observed Value / Detail
Elastic Query Filter	event.dataset: apache.access AND url.query: "UNION" or "SELECT" or "INTO" or "AND" or "OR"
Target Endpoint	/database/?value=alice_wonder[PAYLOAD]
Injection Technique	UNION-based SQL Injection with column count enumeration
Database Targeted	information_schema (tables and columns)
Specific Table Targeted	login (column name extraction confirmed)
HTTP Status	200 OK — server responded successfully to injected queries
Response Size	1091 bytes — consistent data returned across payloads
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36

Payload Encoding	URL-encoded (%27 = apostrophe, %20 = space) — decoded via CyberChef
SQL Comment Used	-- (double dash) to terminate original query after injection
Agent Ephemeral ID	543dc45c-a46b-4e11-ac02-585ef6898638
Agent ID	28533ccf-61c4-47fe-ae31-5b84ccfbc5b4

Attack Timeline

Timestamp	Event
Apr 28, 2025 @ 16:53:29.000	First UNION SELECT payload — NULL column count probe (4 NULLs) — HTTP 200
Apr 28, 2025 @ 16:53:32.000	Second UNION SELECT payload — NULL column count probe (3 NULLs) — HTTP 200
Apr 28, 2025 @ 16:53:48.000	Third payload — union select table_name from information_schema.tables — schema enumeration begins
Apr 28, 2025 @ 16:53:56.000	Fourth payload — repeated table enumeration request — confirmed schema dump
Apr 28, 2025 @ 16:54:07.000	Fifth payload — UNION SELECT column_name FROM information_schema.columns WHERE table_name='login' — login table columns extracted
Apr 28, 2025 @ 16:53:33.367	Events created in Filebeat on host
Apr 28, 2025 @ 16:53:58.373	Events ingested into Elastic SIEM

Evidence & Screenshots

Figure 4 — CyberChef: URL-Decoded SQL Injection Payload

CyberChef was used to decode the URL-encoded payload found in the Apache access logs. The decoded output clearly reveals the UNION SELECT SQL injection syntax targeting information_schema.columns for the login table. The HTTP/1.1 200 response code and 1091-byte response size confirm the server returned data successfully.

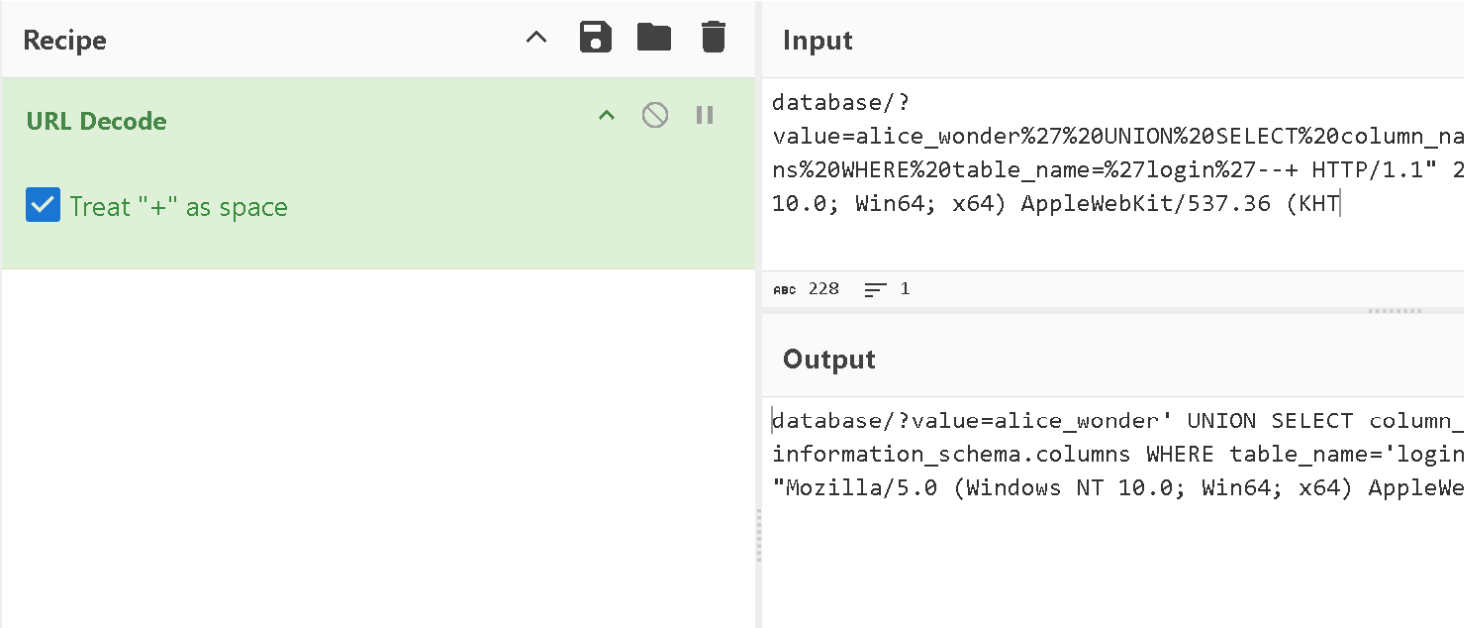


Figure 4: CyberChef URL Decode — revealing UNION SELECT column_name FROM information_schema.columns WHERE table_name='login'

Figure 5 — Elastic SIEM: UNION SELECT Events in Apache Access Logs

The Elastic SIEM query filtering for SQL keywords (UNION, SELECT, INTO, AND, OR) in url.query returned 5 matching documents. Each event clearly shows progressive UNION SELECT payloads with alice_wonder as the injected base value. The events span from 16:53:29 to 16:54:07, all returning HTTP 200 responses — confirming successful server-side execution of the injected SQL queries.

event.dataset: apache.access and url.query: "UNION" or "SELECT" or "INTO" or "AND" or "OR"

Documents (5) Field statistics

Summary

event.dataset apache.access url.query value=alice_wonder' UNION SELECT NULL,NULL,NULL--+ @timestamp Apr 2
0 agent.ephemeral_id 543dc45c-a46b-4e11-ac02-585ef6898638 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 a
e agent.type filebeat agent.version 8.18.0 ecs.version 1.12.0 event.category web event.created Apr 28, 20

event.dataset apache.access url.query value=alice_wonder' union select table_name,2,3 from information_sc
6:53:48.000 agent.ephemeral_id 543dc45c-a46b-4e11-ac02-585ef6898638 agent.id 28533ccf-61c4-47fe-ae31-5b84
e agent.type filebeat agent.version 8.18.0 ecs.version 1.12.0 event.category web event.created Apr 28, 20

event.dataset apache.access url.query value=alice_wonder' union select table_name,2,3 from information_sc
6:53:56.000 agent.ephemeral_id 543dc45c-a46b-4e11-ac02-585ef6898638 agent.id 28533ccf-61c4-47fe-ae31-5b84
e agent.type filebeat agent.version 8.18.0 ecs.version 1.12.0 event.category web event.created Apr 28, 20

event.dataset apache.access url.query value=alice_wonder' UNION SELECT NULL,NULL,NULL,NULL--+ @timestamp
0 agent.ephemeral_id 543dc45c-a46b-4e11-ac02-585ef6898638 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 a
e agent.type filebeat agent.version 8.18.0 ecs.version 1.12.0 event.category web event.created Apr 28, 20

event.dataset apache.access url.query value=alice_wonder' UNION SELECT column_name,2,3 FROM information_s
+ @timestamp Apr 28, 2025 @ 16:54:07.000 agent.ephemeral_id 543dc45c-a46b-4e11-ac02-585ef6898638 agent.id
4 agent.name btweb-virtual-machine agent.type filebeat agent.version 8.18.0 ecs.version 1.12.0 event.cate

Figure 5: Elastic SIEM — 5 SQL UNION injection events detected in Apache access logs on btweb-virtual-machine

Figure 6 — Elastic Field Statistics: Top 50 url.original.keyword Values

The Elastic Field Statistics view for url.original.keyword exposes the top URL values observed, listing all four distinct UNION SELECT payloads in URL-encoded form. This view confirms the complete set of injection strings used by the attacker, spanning NULL-based column probing through to targeted column extraction from the login table.

The first step in this process is to extract the URL queries associated with the previously iden

Top 50 values of url.original.keyword
/database/?value=alice_wonder%27%20union%20select%20table_name,2,3%20from%20information_schema.tables%20--+
/database/?value=alice_wonder%27%20UNION%20SELECT%20NULL,NULL,NULL,NULL--+
/database/?value=alice_wonder%27%20UNION%20SELECT%20NULL,NULL,NULL--+
/database/?value=alice_wonder%27%20UNION%20SELECT%20column_name,2,3%20FROM%20information_schema.columns%2

Figure 6: Elastic top url.original.keyword values — all four URL-encoded UNION SELECT payloads targeting /database/ endpoint

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Collection	**T1213**	Data from Information Repositories
Credential Access	**T1552**	Unsecured Credentials
Discovery	**T1082**	System Information Discovery
Exfiltration	**T1041**	Exfiltration Over C2 Channel

Recommendations & Remediation

The following controls are critical to prevent and mitigate SQL injection attacks:

- Immediately audit and patch the /database/ endpoint — use parameterized queries (prepared statements) for all database interactions to eliminate SQL injection risk entirely.
- Deploy input validation and sanitization on all user-supplied parameters — reject or escape single quotes, SQL keywords, and comment sequences (--).
- Implement a Web Application Firewall (WAF) with SQL injection detection rules to block UNION SELECT, information_schema, and similar patterns in URL parameters.
- Restrict database user privileges — the application database account should not have SELECT access to information_schema tables in production.
- Rotate all credentials stored in the login table immediately, as column names (and potentially values) have been extracted by the attacker.
- Enable Elastic SIEM alerting for SQL keywords (UNION, SELECT, information_schema) appearing in Apache url.query fields.
- Review all HTTP 200 responses to the /database/ endpoint for unexpected data sizes that may indicate successful data extraction.
- Conduct a full database audit to assess the scope of data potentially accessed or exfiltrated during this incident.

ATTACK 04

Investigating SQLMap Attack Detection

Attack Overview

Attack ID	SO-WEB-04
Attack Type	Automated SQL Injection — SQLMap Tool-Based Attack
Severity	Critical
Detection Source	Elastic SIEM — Apache Access Logs
Target Host	btweb-virtual-machine
Target Endpoint	/database/
Injection Parameter	value
Injection Base Value	john_doe
Tool Identified	sqlmap/1.8.5#stable (https://sqlmap.org)
Additional UA Patterns	*python-requests*, *curl* (included in detection filter)
HTTP Response Code	200 OK (successful server responses to injected queries)
Response Size	1028 bytes
Timestamp (Range)	Apr 28, 2025 @ 17:13:46.000 – 17:14:xx.000
Total Events	70 matching documents detected
Agent Ephemeral ID	4fe665be-db61-4665-bbc3-7297ac5483d5
Agent ID	28533ccf-61c4-47fe-ae31-5b84ccfbc5b4
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0
ECS Version	1.12.0

Attack Description

SQLMap is an open-source penetration testing tool that automates the detection and exploitation of SQL injection vulnerabilities. Unlike manual injection (as seen in Attack 03), SQLMap conducts a comprehensive, automated battery of injection techniques — including boolean-based blind, time-based blind, error-based, UNION-based, and stacked queries — to fully compromise a vulnerable database endpoint. It identifies injectable parameters, determines the database type, and can autonomously dump tables, extract credentials, and even execute operating system commands.

In this incident, the attacker ran SQLMap version 1.8.5 (stable) against the /database/ endpoint using the value parameter (base value: john_doe). The tool generated 70 distinct HTTP requests containing advanced SQL payloads. The Elastic SIEM query — filtering for SQL keywords in

url.query combined with User-Agent patterns for sqlmap, python-requests, and curl — returned all 70 events. The User-Agent string sqlmap/1.8.5#stable (https://sqlmap.org) is present across all events, making attribution to SQLMap unambiguous.

The detected payloads demonstrate SQLMap operating across multiple injection techniques simultaneously. Error-based techniques using ROW() and FLOOR(RAND()*2) are used to trigger database errors that leak data. BENCHMARK() and EXP() functions are leveraged for time-based blind injection to infer data through response delays. UNION ALL SELECT payloads with hex-encoded strings (0x71...) extract data by appending results to legitimate query output. All injections returned HTTP 200 with a consistent 1028-byte response, confirming repeated successful data retrieval.

SQLMap Injection Techniques Observed

Multiple advanced SQL injection techniques were identified across the 70 captured events:

Technique	Payload Indicator Observed	Purpose
Error-Based (ROW)	`OR ROW(x,y)>(SELECT COUNT(*),CONCAT(...),FLOOR(RAND(0)*2))x FROM ...`	Triggers DB error to leak data values in error messages
Error-Based (GTID)	`OR GTID_SUBSET(CONCAT(0x71..., SELECT ELT(x=x,1)), x)`	Uses GTID function to extract data via MySQL error output
Time-Based Blind	`SELECT BENCHMARK(5000000, MD5(0x4b6a726c))`	Causes deliberate delay to infer boolean conditions without visible output
Error-Based (EXP)	`OR EXP(~(SELECT * FROM (SELECT CONCAT(...)) x))`	Overflow-based technique to extract data through MySQL error
UNION ALL SELECT	`UNION ALL SELECT NULL,NULL,CONCAT(0x71...,0x48526c...),0x71706a7171`	Appends data from DB to query response using hex-encoded delimiters
Stacked UNION Chain	`UNION SELECT 0 AS iaWw UNION SELECT 1 UNION SELECT 2 ... UNION SELECT 14`	Column count determination using extended UNION SELECT chain

Technical Analysis

Indicator	Observed Value / Detail
Elastic Query Filter	event.dataset: apache.access AND url.query: ("UNION" or "SELECT" or "INTO" or "AND" or "OR") AND user_agent.original: (*sqlmap* or *python-requests* or *curl*)
Total Matching Documents	70 events — highest volume of any attack in this chain
Primary Tool	sqlmap/1.8.5#stable (https://sqlmap.org)
Injection Base Value	john_doe (substituted with multiple advanced payloads)
HTTP Status Code	200 OK — all injection attempts returned successful responses
Response Size	1028 bytes — consistent across events indicating stable data return
Hex Encoded Delimiters	0x7171626b71 and 0x71706a7171 — SQLMap boundary markers for

	data extraction
SQL Functions Used	BENCHMARK(), MD5(), FLOOR(RAND(0)*2), CONCAT(), ELT(), GTID_SUBSET(), EXP()
Comment Style	# (hash) and -- (double dash) used to terminate original queries
Injection Vector	GET parameter value in /database/ endpoint
Encoding	URL-encoded payloads decoded via CyberChef to confirm syntax
Attack Duration	~1 minute sustained automated scan (17:13:46 – 17:14:xx)

Attack Timeline

Timestamp	Event
Apr 28, 2025 @ 17:13:46.000	SQLMap scan begins — EXP() error-based blind injection — HTTP 200, 1028 bytes
Apr 28, 2025 @ 17:13:48.000	GTID_SUBSET error-based payload fired — automated progression through techniques
Apr 28, 2025 @ 17:13:55.000	ROW() error-based injection with FLOOR(RAND(0)*2) — multiple UNION SELECT chains embedded
Apr 28, 2025 @ 17:13:56.000	UNION ALL SELECT with hex-encoded delimiters — data extraction phase begins
Apr 28, 2025 @ 17:14:xx.000	BENCHMARK(5000000) time-based blind payload — confirming time-based inference capability
Apr 28, 2025 (sustained)	70 total requests fired automatically — full SQLMap test battery executed against endpoint

Evidence & Screenshots

Figure 7 — Elastic SIEM: 70 SQLMap Events in Apache Access Logs

The Elastic SIEM query filters Apache access logs for SQL keywords combined with sqlmap, python-requests, and curl User-Agent patterns. 70 matching documents are returned. Each event shows the sqlmap/1.8.5#stable User-Agent string and complex automated injection payloads in the url.query field. Events span multiple SQLMap techniques executed in rapid succession against the /database/ endpoint.

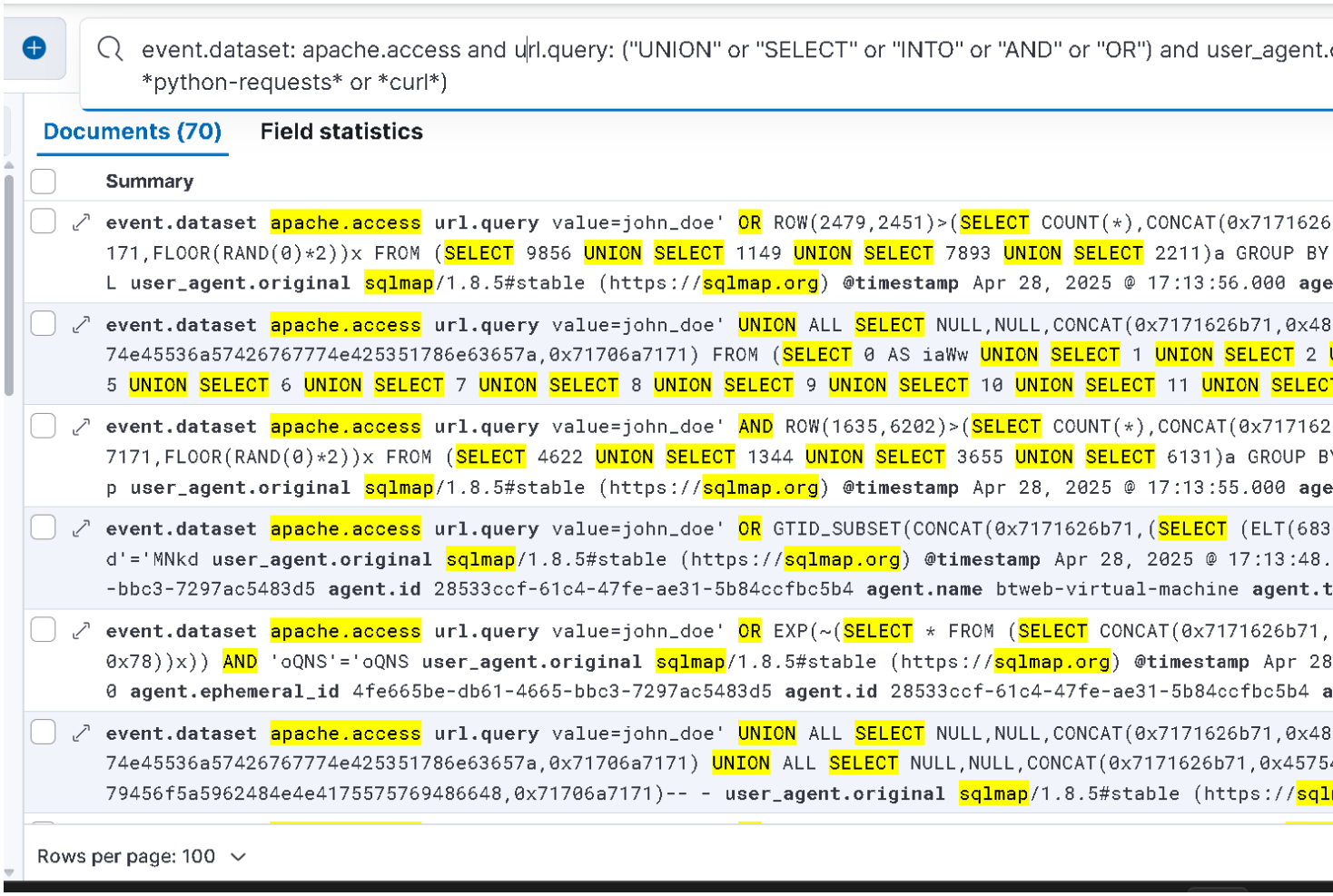


Figure 7: Elastic SIEM — 70 SQLMap automated injection events with sqlmap/1.8.5 User-Agent across multiple SQL techniques

Figure 8 — CyberChef: URL-Decoded SQLMap Payloads

CyberChef URL Decode was applied to the raw Apache log entries, revealing the full decoded SQLMap payload syntax. The output confirms two distinct payloads: a BENCHMARK(5000000, MD5()) time-based blind injection and a ROW()-based error injection with nested UNION SELECT chains. Both returned HTTP 200 with 1028 bytes, confirming the server executed and responded to all injected SQL statements.

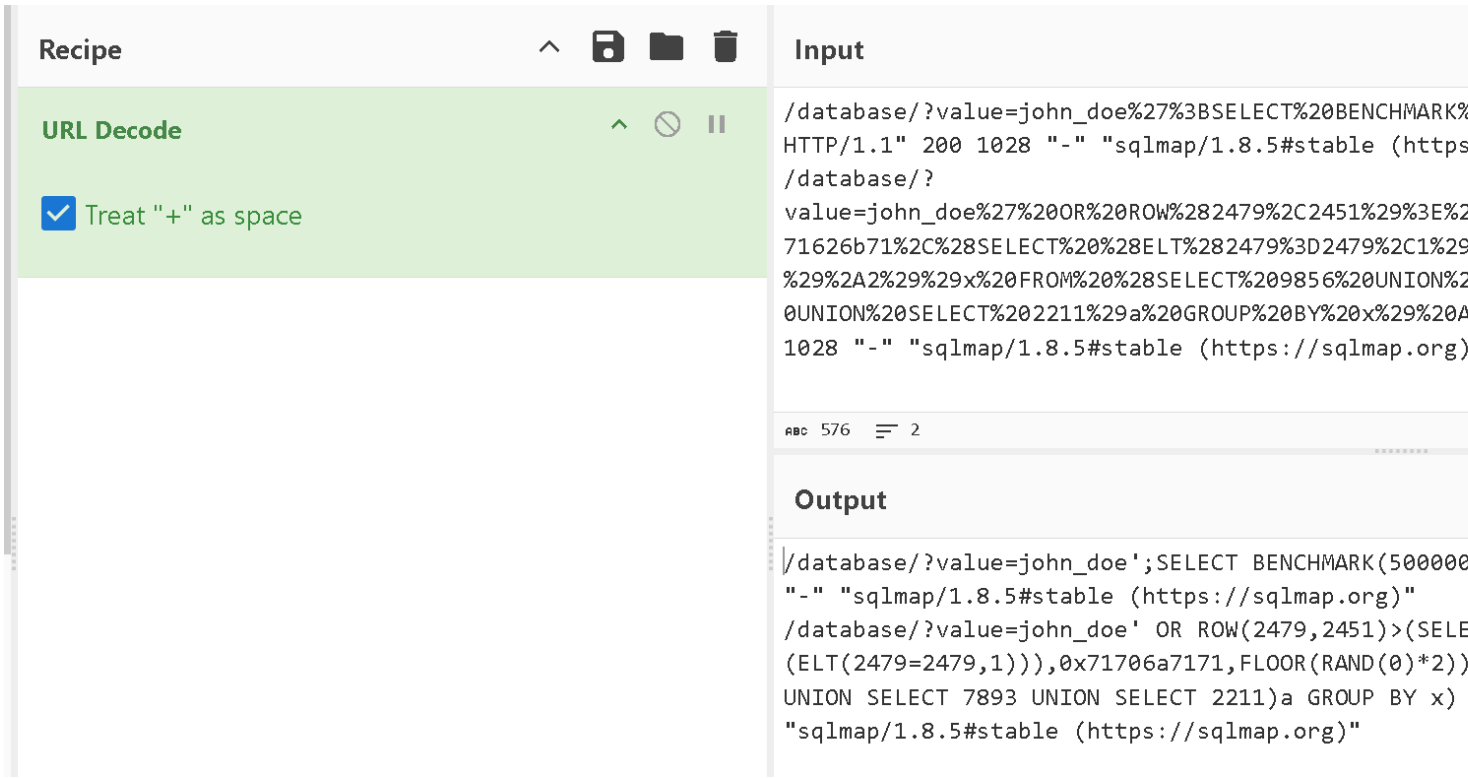


Figure 8: CyberChef URL Decode — BENCHMARK() time-based blind and ROW() error-based SQLMap payloads decoded from Apache logs

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Execution	**T1059.007**	Command and Scripting Interpreter: JavaScript / Python
Collection	**T1213**	Data from Information Repositories
Credential Access	**T1552.001**	Credentials In Files
Discovery	**T1082**	System Information Discovery
Exfiltration	**T1048**	Exfiltration Over Alternative Protocol

Recommendations & Remediation

The following controls are recommended to prevent and detect automated SQLMap-based attacks:

- Block the sqlmap User-Agent string at the WAF or reverse proxy level immediately — this is a reliable and unambiguous indicator of automated attack tooling.
- Implement parameterized queries (prepared statements) for all database interactions — this is the single most effective defense against all SQL injection variants.

- Deploy a WAF with advanced SQLi rules to detect and block error-based, time-based, and UNION-based injection patterns in HTTP parameters.
- Apply strict input validation on the value parameter — reject requests containing SQL functions (BENCHMARK, FLOOR, RAND, CONCAT, EXP, UNION, SELECT).
- Enforce rate limiting: block or alert on any source IP sending more than 10 requests per minute to the /database/ endpoint.
- Rotate all database credentials immediately — SQLMap with 70 successful HTTP 200 responses is highly likely to have extracted data.
- Investigate and forensically examine the database for unauthorized reads, dumps, or schema traversals during the 17:13–17:14 window.
- Consider disabling or restricting the /database/ endpoint from public access and placing it behind authentication.

ATTACK 05

Investigating Directory Traversal Attack

Attack Overview

Attack ID	SO-WEB-05
Attack Type	Directory Traversal / Path Traversal (LFI)
Severity	Critical
Detection Source	Elastic SIEM — Apache Access Logs + Wireshark PCAP
Target Host	btweb-virtual-machine
Target IP	172.16.25.15
Source IP	10.20.10.3
Target Endpoint	/career/pdfresum.php
Injection Parameter	file
Traversal Sequence	../../../../../../ (varying depths)
Target System Files	/etc/passwd, /etc/shadow, /etc/hosts, /etc/mysql/my.cnf, /etc/issue
HTTP Response (Wireshark)	302 Found — server redirected on successful traversal attempts
HTTP Response (Elastic)	No status filter — all traversal events captured
Timestamp (First)	May 2, 2025 @ 12:00:53.000
Timestamp (Last)	May 2, 2025 @ 12:02:25.000
Total Events	7 matching documents in Elastic SIEM
Agent Ephemeral ID	cbe90fe8-f9d5-4f37-84a4-f86eb4d4a4f3
Agent ID	28533ccf-61c4-47fe-ae31-5b84ccfbc5b4
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0
ECS Version	1.12.0

Attack Description

A Directory Traversal attack (also known as Path Traversal or Local File Inclusion — LFI) exploits insufficient input validation on file path parameters in a web application. By injecting sequences of ../ (dot-dot-slash), the attacker navigates outside the intended web root directory and accesses arbitrary files on the underlying server filesystem. This attack requires no authentication and can expose highly sensitive system files including password hashes, SSH keys, database configurations, and system logs.

In this incident, the attacker targeted the /career/pdfresum.php endpoint, which accepts a file parameter intended to load PDF resume files. By substituting legitimate file paths with traversal sequences of varying depth (../../../../etc/ through ../../../../../../../../etc/), the attacker systematically attempted to read critical Linux system files. The target files indicate a deliberate, structured attack to gather credentials and system configuration data: /etc/passwd contains user account information, /etc/shadow holds hashed passwords, /etc/hosts reveals the network configuration, and /etc/mysql/my.cnf exposes MySQL database credentials and configuration.

Network packet capture in Wireshark corroborates the Elastic findings. The PCAP filter http.request.uri contains "../../../../../../" reveals the raw GET requests from 10.20.10.3 to 172.16.25.15 with traversal sequences in the file parameter. The server responded with HTTP 302 Found (redirect) for several of these requests, which in LFI contexts can indicate the application processed the path and redirected rather than serving the file directly — a behavior that may still confirm file path processing occurred server-side.

Traversal Payloads Identified

The following traversal payloads were detected across both Elastic SIEM and Wireshark capture:

#	Traversal Payload (file parameter)	Target File	Timestamp
1	`file=../../../../etc/issue`	**/etc/issue**	12:00:53.000
2	`file=../../../../etc/passwd`	**/etc/passwd**	12:01:01.000
3	`file=../../../../etc`	**/etc (directory)**	12:01:17.000
4	`file=../../../../../../../../etc/passwd`	**/etc/passwd**	12:01:40.000
5	`file=../../../../../../../../etc/shadow`	**/etc/shadow**	12:01:53.000
6	`file=../../../../../../../../etc/hosts`	**/etc/hosts**	12:02:16.000
7	`file=../../../../../../../../etc/mysql/my.cnf`	**/etc/mysql/my.cnf**	12:02:25.000

Target File Significance

File Targeted	Sensitive Data Exposed / Risk
`/etc/passwd`	All system user accounts, home directories, and shell assignments — user enumeration
`/etc/shadow`	Hashed passwords for all system users — offline cracking risk is extremely high
`/etc/hosts`	Internal network hostname-to-IP mappings — exposes internal network topology
`/etc/mysql/my.cnf`	MySQL database configuration including credentials, bind address, and socket paths

`/etc/issue`	OS banner and version string — confirms OS type/version for further exploitation
`/etc (directory)`	Directory listing attempt — mapping the /etc directory structure for further traversal

Technical Analysis

Indicator	Observed Value / Detail
Elastic Query Filter	event.dataset: apache.access AND url.query.keyword: *../../../../* AND url.query: *etc*
Wireshark Filter	http.request.uri contains "../../../../../"
Vulnerable Endpoint	/career/pdfresum.php?file=[traversal_payload]
Traversal Depth	4 levels (../../../../) to 8 levels (../../../../../../../../)
HTTP Method	GET — traversal delivered via URL query parameter
Wireshark Response Codes	302 Found — server processed path and redirected
Source IP	10.20.10.3
Destination IP	172.16.25.15 (port 80)
Attack Duration	~92 seconds (12:00:53 to 12:02:25)
Files Successfully Requested	7 distinct sensitive file paths targeted
Attack Pattern	Incrementally deeper traversal sequences — methodical manual or scripted approach
Agent Ephemeral ID	cbe90fe8-f9d5-4f37-84a4-f86eb4d4a4f3

Attack Timeline

Timestamp	Event
May 2, 2025 @ 12:00:53.000	First traversal — file=../../../../etc/issue — OS banner targeted
May 2, 2025 @ 12:01:01.000	file=../../../../etc/passwd — user account enumeration attempt
May 2, 2025 @ 12:01:17.000	file=../../../../etc — directory listing attempt to map /etc structure
May 2, 2025 @ 12:01:40.000	Deeper traversal: file=../../../../../../../../etc/passwd — increased depth
May 2, 2025 @ 12:01:53.000	file=../../../../../../../../etc/shadow — password hash extraction attempt
May 2, 2025 @ 12:02:16.000	file=../../../../../../../../etc/hosts — internal network topology probing
May 2, 2025 @ 12:02:25.000	file=../../../../../../../../etc/mysql/my.cnf — DB credentials targeted
May 2, 2025 @ 12:12:37.785	Events created in Filebeat on host (10-min pipeline delay)
May 2, 2025 @ 12:12:48.955	Events ingested into Elastic SIEM

Evidence & Screenshots

Figure 9 — Elastic SIEM: Directory Traversal Events in Apache Access Logs

The Elastic SIEM query filtering for ../../../../* traversal sequences combined with *etc* in url.query returned 7 documents. Each event clearly shows the file parameter carrying a traversal payload targeting a different sensitive file under /etc/. The incrementally deeper sequences (4-level to 8-level traversal) confirm a methodical approach to bypassing any depth-based path restrictions on the server.

event.dataset: apache.access and url.query.keyword: *../../../../* and url.query: *etc*

Documents (7) Field statistics

Summary

```
event.dataset apache.access url.query file=../../../../etc/issue @timestamp May 2, 2025 @ 12:00:53.000 age
f86eb4d4a4f3 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 agent.name btweb-virtual-machine agent.type fil
0 ecs.version 1.12.0 event.category web event.created May 2, 2025 @ 12:12:37.785 event.ingested May 2, 202

event.dataset apache.access url.query file=../../../../etc/passwd @timestamp May 2, 2025 @ 12:01:01.000 ag
-f86eb4d4a4f3 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 agent.name btweb-virtual-machine agent.type fi
0 ecs.version 1.12.0 event.category web event.created May 2, 2025 @ 12:12:37.785 event.ingested May 2, 202

event.dataset apache.access url.query file=../../../../etc @timestamp May 2, 2025 @ 12:01:17.000 agent.eph
d4a4f3 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 agent.name btweb-virtual-machine agent.type filebeat
0 event.category web event.created May 2, 2025 @ 12:12:37.785 event.ingested May 2, 2025 @ 12:12:48.956 ev

event.dataset apache.access url.query file=../../../../../../../../etc/passwd @timestamp May 2, 2025 @ 12
d5-4f37-84a4-f86eb4d4a4f3 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 agent.name btweb-virtual-machine a
0 ecs.version 1.12.0 event.category web event.created May 2, 2025 @ 12:12:37.785 event.ingested May 2, 202

event.dataset apache.access url.query file=../../../../../../../../etc/shadow @timestamp May 2, 2025 @ 12
d5-4f37-84a4-f86eb4d4a4f3 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 agent.name btweb-virtual-machine a
0 ecs.version 1.12.0 event.category web event.created May 2, 2025 @ 12:12:37.785 event.ingested May 2, 202

event.dataset apache.access url.query file=../../../../../../../../etc/hosts @timestamp May 2, 2025 @ 12:0
5-4f37-84a4-f86eb4d4a4f3 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 agent.name btweb-virtual-machine ag
0 ecs.version 1.12.0 event.category web event.created May 2, 2025 @ 12:12:37.785 event.ingested May 2, 202

event.dataset apache.access url.query file=../../../../../../../../etc/mysql/my.cnf @timestamp May 2, 2025
fe8-f9d5-4f37-84a4-f86eb4d4a4f3 agent.id 28533ccf-61c4-47fe-ae31-5b84ccfbc5b4 agent.name btweb-virtual-mac
```

Figure 9: Elastic SIEM — 7 directory traversal events targeting /etc/passwd, /etc/shadow, /etc/hosts, /etc/mysql/my.cnf on btweb-virtual-machine

Figure 10 — Wireshark PCAP: Raw GET Requests with Path Traversal Sequences

The Wireshark packet capture with the filter for deep traversal sequences in http.request.uri confirms the raw network-level traversal requests from 10.20.10.3 to 172.16.25.15. The capture shows GET requests to /career/pdfresum.php with 8-level ../ sequences targeting /etc/passwd, /etc/shadow, /etc/hosts, and /etc/mysql/my.cnf. Server responses of HTTP 302 Found indicate the application processed and acted on the traversal paths.

http.request.uri contains "../../../../../../../"

No.	Time	Source	Destination	Protocol	Length	Info
296	125.214175216	10.20.10.3	172.16.25.15	HTTP	501	GET /career/pdfresum.php?file=../../../..
299	125.216367161	172.16.25.15	10.20.10.3	HTTP	584	HTTP/1.1 302 Found (text/html)
310	138.138906595	10.20.10.3	172.16.25.15	HTTP	501	GET /career/pdfresum.php?file=../../../..
312	138.139969990	172.16.25.15	10.20.10.3	HTTP	368	HTTP/1.1 302 Found (text/html)
320	161.293582774	10.20.10.3	172.16.25.15	HTTP	500	GET /career/pdfresum.php?file=../../../..
322	161.294813863	172.16.25.15	10.20.10.3	HTTP	604	HTTP/1.1 302 Found (text/html)
330	169.882009865	10.20.10.3	172.16.25.15	HTTP	507	GET /career/pdfresum.php?file=../../../..
332	169.885109502	172.16.25.15	10.20.10.3	HTTP	1057	HTTP/1.1 302 Found (text/html)

Figure 10: Wireshark PCAP — GET /career/pdfresum.php with 8-level traversal payloads targeting /etc/ system files, server responding HTTP 302*

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Discovery	**T1083**	File and Directory Discovery
Credential Access	**T1552.001**	Credentials in Files
Collection	**T1005**	Data from Local System
Exfiltration	**T1041**	Exfiltration Over C2 Channel

Recommendations & Remediation

The following controls are critical to prevent directory traversal and local file inclusion attacks:

- Immediately patch the /career/pdfresum.php endpoint — never pass user-supplied file paths directly to filesystem functions; use an allowlist of permitted file names or IDs instead.
- Implement strict server-side input validation: reject any file parameter containing ../, %2e %2e/, or encoded variants before processing.
- Use realpath() or equivalent canonicalization functions to resolve the absolute path and verify it falls within the intended web root directory before serving any file.
- Restrict the web server process user permissions — Apache should not have read access to /etc/shadow, /etc/mysql/, or other sensitive system directories.
- Rotate all credentials that may have been exposed: Linux system user passwords, MySQL database credentials, and any credentials in /etc/mysql/my.cnf.
- Configure Elastic SIEM alerts to trigger on any url.query containing traversal sequences (../) combined with sensitive paths (/etc/, /var/, /home/).
- Apply Apache mod_security rules to detect and block path traversal patterns in request parameters.
- Conduct a full forensic review to determine whether /etc/shadow contents were successfully retrieved and whether any offline password cracking has been attempted.

ATTACK 06

Investigating Classic SQL Injection Detection

Attack Overview

Attack ID	SO-WEB-06
Attack Type	Classic SQL Injection — Authentication Bypass (Login Form)
Severity	Critical
Detection Source	Elastic SIEM — Apache Access Logs + Wireshark PCAP
Target Host	btweb-virtual-machine
Target IP	172.16.25.15
Source IP	10.20.10.3
Target Endpoint	/login/
HTTP Method	POST
Content-Type	application/x-www-form-urlencoded
SQLi Payload (username)	' OR 1=1 #
Password Supplied	asdasd (arbitrary — bypassed by injection)
Failed Login Events	18 documents — event.outcome: failure
Successful Login Events	2 documents — event.outcome: success, HTTP 200
First Failed Attempt	May 5, 2025 @ 15:42:46.000
Successful Login #1	May 5, 2025 @ 15:42:25.000
Successful Login #2	May 5, 2025 @ 15:44:44.000
Agent Ephemeral ID	ff6bbef0-7b0e-4856-8bfb-5ddb42e20751
Agent ID	28533ccf-61c4-47fe-ae31-5b84ccfbc5b4
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0
ECS Version	1.12.0

Attack Description

Classic SQL Injection authentication bypass is one of the most fundamental and devastating web application attacks. By injecting malicious SQL syntax into a login form's username field, the attacker manipulates the underlying SQL query so that it evaluates to TRUE regardless of the password supplied. The canonical payload ' OR 1=1 # works by closing the username string with a single quote, appending an always-true condition (OR 1=1), and using the hash (#) character to

comment out the remainder of the original SQL query — including the password check — effectively granting access without valid credentials.

In this incident, the attacker targeted the /login/ endpoint on the web application with a sustained series of POST requests. The Wireshark packet capture reveals the decoded HTML form data submitted in one of these requests: username = ' OR 1=1 # and password = asdasd. Despite the arbitrary password, the SQL injection in the username field caused the authentication query to bypass credential validation entirely. The attack sequence began at 15:42:25 with an initial successful login, was followed by 18 failure events as the attacker tested variations, and concluded with a second confirmed successful authentication at 15:44:44.

The two confirmed successful authentications (HTTP 200, event.outcome: success) represent a critical security breach — the attacker gained unauthorized access to the web application without knowing any valid credentials. Once authenticated, the attacker would have access to all functionality available to a legitimate logged-in user, including any administrative features, data retrieval endpoints, and further exploitation vectors within the application.

SQL Injection Payload Analysis

The classic authentication bypass payload was extracted directly from Wireshark form data inspection:

Form Field	Submitted Value	Effect / Analysis
username	`' OR 1=1 #`	Closes string, appends always-true condition, comments out password check — authentication bypassed
password	`asdasd`	Arbitrary value — irrelevant due to SQL injection bypassing the password validation entirely
login	`Login`	Standard form submission button value — confirms this is a standard login form POST request

How the injected SQL query evaluates on the server:

Original Query	`SELECT * FROM users WHERE username='[input]' AND password='[input]'`
Injected Query	`SELECT * FROM users WHERE username='' OR 1=1 #' AND password='asdasd'`
Result	Password check is commented out. 1=1 is always TRUE. Query returns all users — first user row (typically admin) is authenticated.

Technical Analysis

Indicator	Observed Value / Detail

Elastic Query (Failures)	event.dataset: apache.access AND url.path.keyword: "/login/" AND event.outcome.keyword: "failure"
Elastic Query (Success)	event.dataset: apache.access AND url.path.keyword: "/login/" AND event.outcome.keyword: "success" AND http.response.status_code: "200"
Wireshark Filter	http.request.uri == "/login/" and http.request.method == "POST"
Failed Attempts	18 events — attacker testing payload variations before successful bypass
Successful Authentications	2 events — HTTP 200, event.outcome: success
First Success Timestamp	May 5, 2025 @ 15:42:25.000 (before failure burst)
Second Success Timestamp	May 5, 2025 @ 15:44:44.000 (after failure burst)
Failure Burst Duration	15:42:46 to 15:43:09 — 18 failed attempts in ~23 seconds
Packet Frame (First)	Frame 71 — 692 bytes — POST /login/ HTTP/1.1
TCP Source Port	57568
TCP Destination Port	80
Content-Type	application/x-www-form-urlencoded (standard HTML form POST)

Attack Timeline

Timestamp	Event
May 5, 2025 @ 15:42:25.000	**SUCCESSFUL LOGIN — HTTP 200 — SQLi authentication bypass confirmed (event.outcome: success)**
May 5, 2025 @ 15:42:46.000	First failure event — attacker testing payload variants at /login/
May 5, 2025 @ 15:42:50.000	Continued POST failures — 4-second interval between attempts
May 5, 2025 @ 15:42:54.000	Third failure — consistent timing indicates scripted/manual testing
May 5, 2025 @ 15:42:58.000	Fourth failure — username/password combinations being varied
May 5, 2025 @ 15:43:03.000	Fifth failure — ' OR 1=1 # payload variants tested
May 5, 2025 @ 15:43:09.000	Sixth+ failure — 18 total failures across the attack window
May 5, 2025 @ 15:44:44.000	SECOND SUCCESSFUL LOGIN — HTTP 200 — confirmed second bypass
May 5, 2025 @ 15:48:04.767	Events created in Filebeat (event.created timestamp)
May 5, 2025 @ 15:48:18.798	Events ingested into Elastic SIEM

Evidence & Screenshots

Figure 11 — Elastic SIEM: 18 Failed Login Attempts at /login/

The Elastic query filtering for url.path: /login/ with event.outcome: failure returns 18 documents. Events span from 15:42:46 to 15:43:09, each showing the apache.access dataset from the same agent (ff6bbef0-7b0e-4856-8bfb-5ddb42e20751) on btweb-virtual-machine. The consistent 4-

second interval between events indicates a structured, deliberate testing sequence as the attacker refined their SQL injection payload.

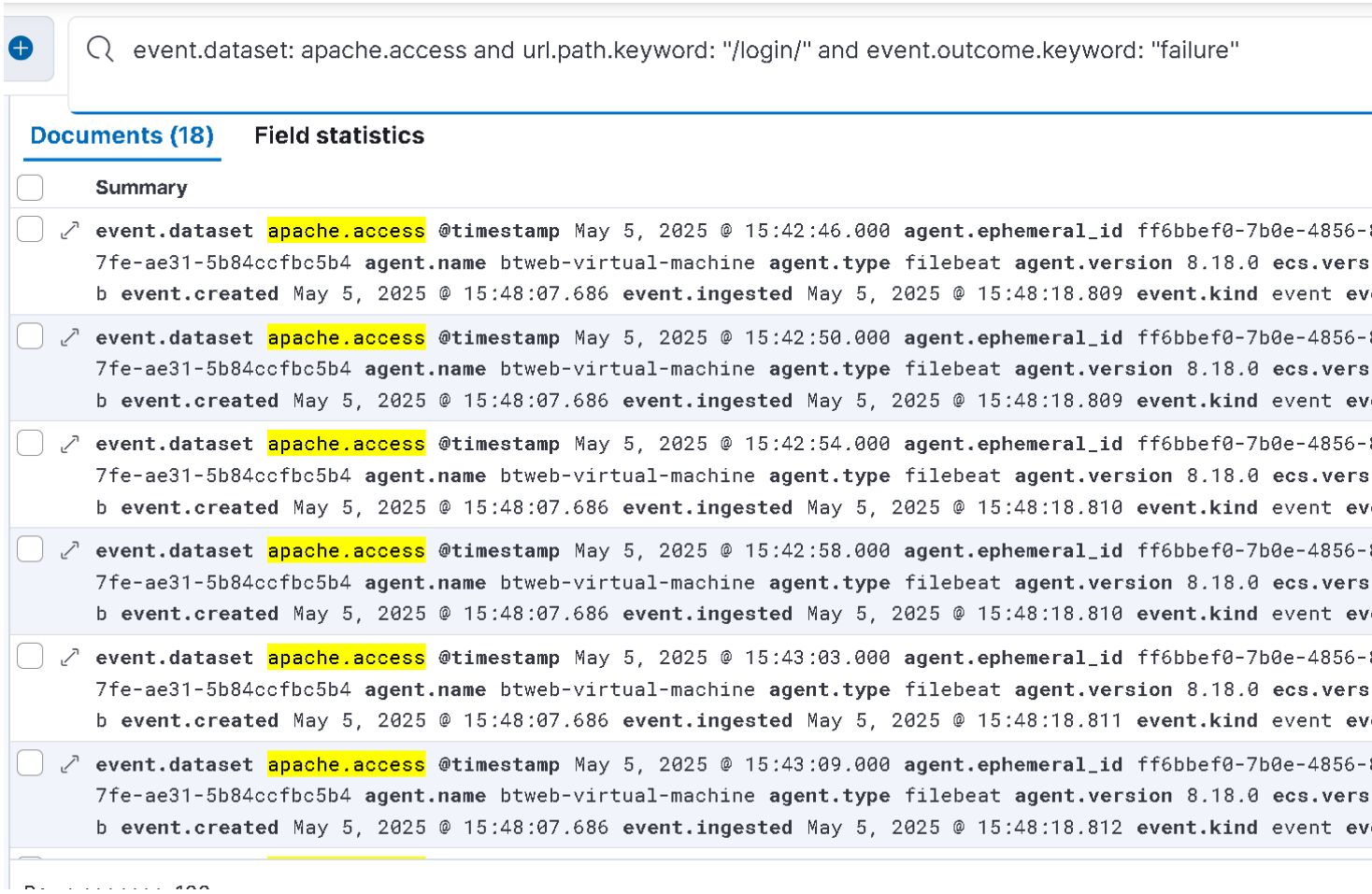


Figure 11: Elastic SIEM — 18 failed POST login attempts to /login/ from 10.20.10.3 on May 5, 2025

Figure 12 — Elastic SIEM: 2 Confirmed Successful Logins (HTTP 200)

The Elastic query for event.outcome: success with HTTP 200 at /login/ returns exactly 2 documents. These represent the two confirmed authentication bypass events — timestamps 15:42:25 and 15:44:44 — flanking the failure burst. Both share the same agent ephemeral ID (ff6bbef0-7b0e-4856-8bfb-5ddb42e20751), confirming they originated from the same session as the failed attempts.

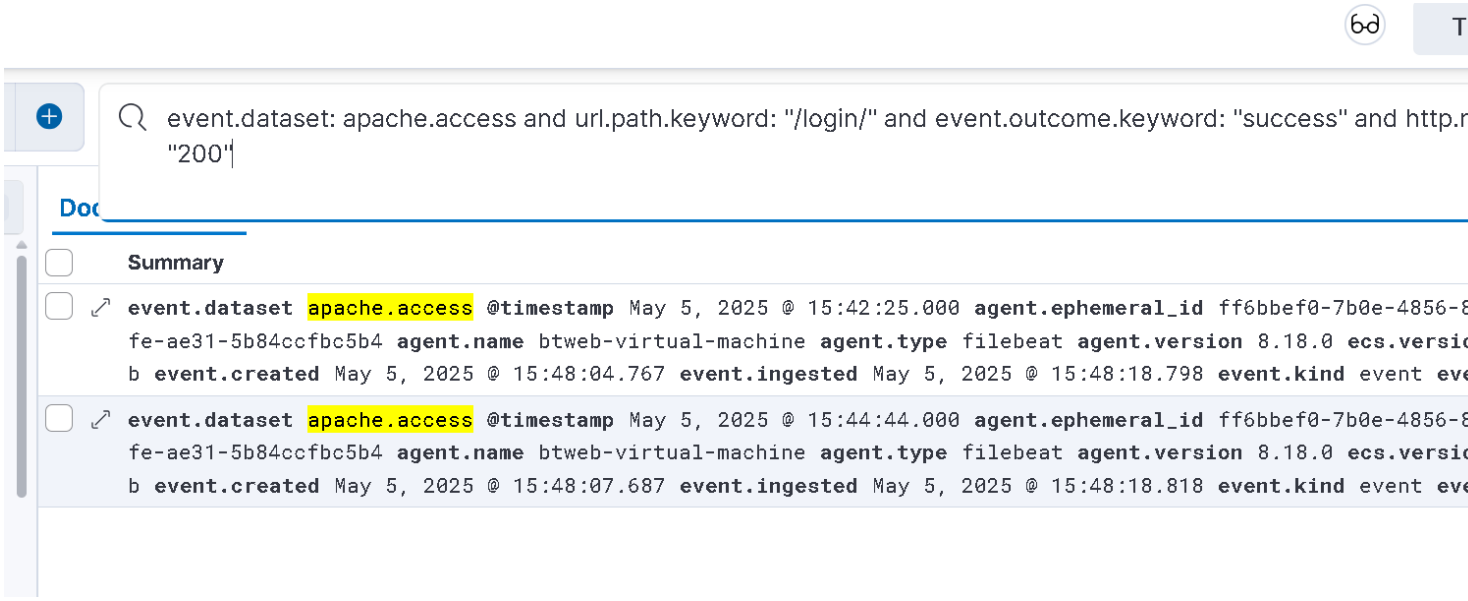


Figure 12: Elastic SIEM — 2 successful HTTP 200 login events confirming SQLi authentication bypass at /login/

Figure 13 — Wireshark PCAP: POST /login/ Requests with Form Data

The Wireshark capture filtered for POST requests to /login/ shows the sustained series of form submissions from 10.20.10.3 to 172.16.25.15:80. Frame 71 is selected (692 bytes), with packet detail confirming HTTP/1.1 POST with application/x-www-form-urlencoded content. The source MAC (VMware_96:ef:ce) and destination MAC (VMware_76:66:76) match the same host pair observed in Attack 01, confirming the same attacker machine throughout the attack chain.

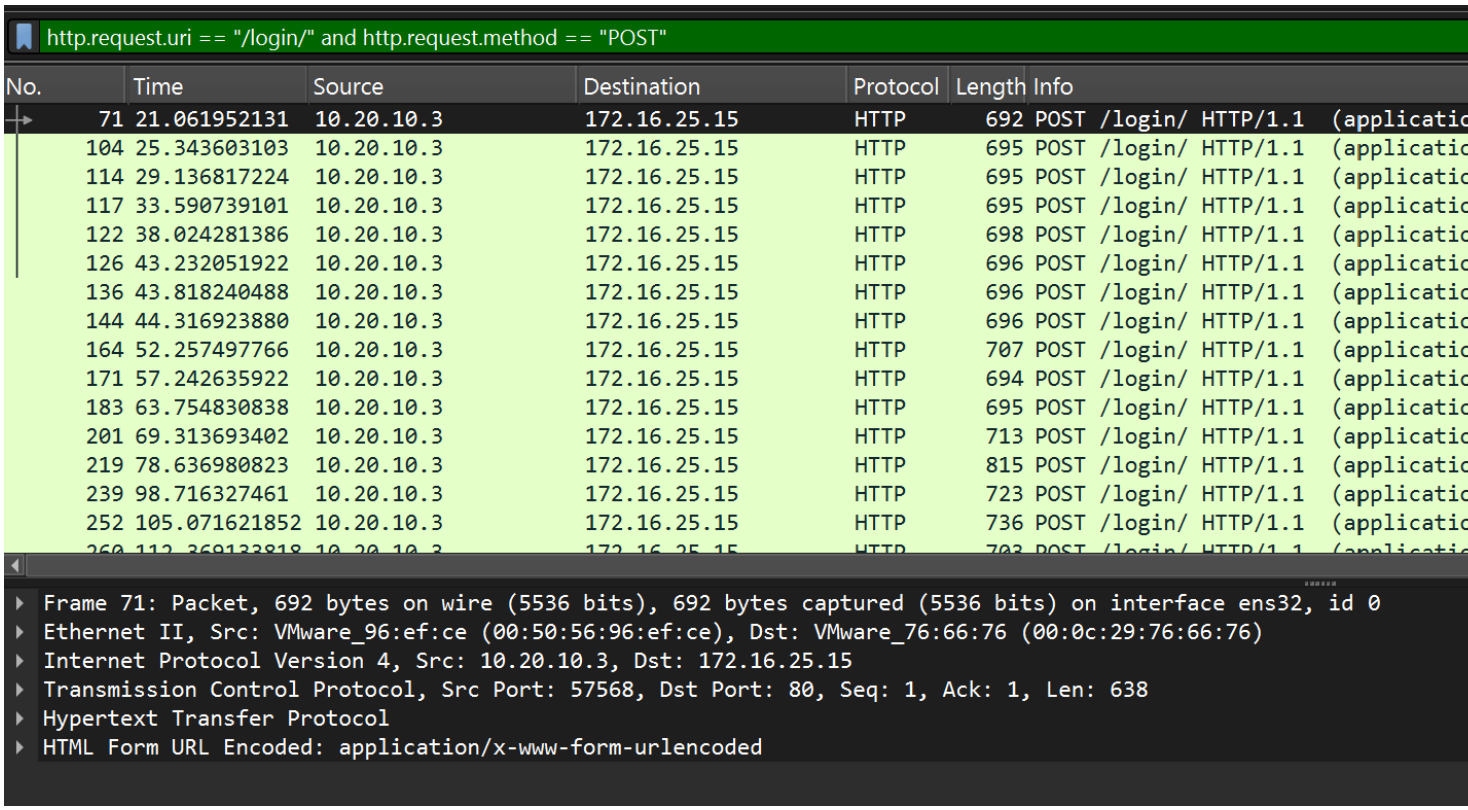


Figure 13: Wireshark PCAP — sustained POST /login/ form submissions from 10.20.10.3 to 172.16.25.15, application/x-www-form-urlencoded

Figure 14 — Wireshark Form Data: SQLi Payload Decoded in HTTP Body

Wireshark's HTML Form URL Encoded dissector reveals the exact form fields transmitted in the POST body. The username field contains the classic SQL injection payload ' OR 1=1 # — confirming the bypass technique used. The password field contains the arbitrary string asdasd, and the login field confirms this is a standard form submission. This is direct, unambiguous evidence of the SQL injection payload captured at the packet level.

```
HTML Form URL Encoded: application/x-www-form-urlenc
  Form item: "username" = "' OR 1=1 #"
    Key: username
    Value: ' OR 1=1 #
  Form item: "password" = "asdasd"
    Key: password
    Value: asdasd
  Form item: "login" = "Login"
    Key: login
    Value: Login
```

Figure 14: Wireshark form data dissector — username: ' OR 1=1 # | password: asdasd — classic SQLi bypass payload captured in HTTP body

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Defense Evasion	**T1078**	Valid Accounts (via authentication bypass)
Credential Access	**T1110.001**	Brute Force: Password Guessing
Persistence	**T1078.003**	Valid Accounts: Local Accounts
Collection	**T1213**	Data from Information Repositories

Recommendations & Remediation

The following controls are essential to prevent classic SQL injection authentication bypass:

- Immediately replace all string-concatenated SQL queries in the authentication logic with parameterized queries (prepared statements) — this eliminates SQL injection at its root.
- Implement an ORM (Object-Relational Mapper) layer to abstract direct SQL construction and prevent injection through user-supplied input.
- Deploy a WAF rule to detect and block payloads containing SQL injection patterns (' OR, 1=1, --, #, UNION SELECT) in POST body form fields.
- Implement account lockout after 5 consecutive failed login attempts — the 18-failure sequence should have triggered an automatic block.
- Add multi-factor authentication (MFA) to the login endpoint so that even a successful authentication bypass does not grant full access.
- Review application session logs for the 15:42:25 and 15:44:44 time windows to determine what actions the attacker performed while authenticated.
- Rotate all application user credentials and session tokens — the attacker gained authenticated access and may have escalated privileges.
- Enable Elastic SIEM alerting for any login endpoint returning HTTP 200 after multiple 4xx failures from the same IP within a 5-minute window.

ATTACK 07

Investigating IDOR Attack

Attack Overview

Attack ID	SO-WEB-07
Attack Type	Insecure Direct Object Reference (IDOR) — Unauthorized Data Access
Severity	High
Detection Source	Elastic SIEM — Apache Access Logs
Target Host	btweb-virtual-machine
Injection Parameter	user_id (GET query parameter)
Values Accessed	user_id=1, user_id=3 (other users' records accessed directly)
HTTP Response Code	200 OK — server returned data for both unauthorized requests
Elastic Query	event.dataset: apache.access AND url.query > 100 AND http.response.status_code: 200
Total Events Detected	2 matching documents
Timestamp (user_id=1)	May 6, 2025 @ 12:18:02.000
Timestamp (user_id=3)	May 6, 2025 @ 12:18:03.000
Event Created	May 6, 2025 @ 12:18:36.270
Event Ingested	May 6, 2025 @ 12:18:47.397 / 12:18:47.398
Agent Ephemeral ID	d5c32d76-2e83-4b42-b1c7-38b3288d6c61
Agent ID	28533ccf-61c4-47fe-ae31-5b84ccfbc5b4
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0
ECS Version	1.12.0

Attack Description

Insecure Direct Object Reference (IDOR) is a type of access control vulnerability that occurs when a web application uses user-controlled input to access objects directly — such as database records, files, or account profiles — without verifying that the requesting user has authorization to access the requested resource. IDOR is consistently ranked among the most critical web application vulnerabilities and is listed in the OWASP Top 10 under Broken Access Control (A01).

In this incident, the attacker manipulated the user_id GET parameter to access records belonging to other users. By simply changing the value of user_id in the URL query string — from their own authenticated ID to user_id=1 and user_id=3 — the attacker was able to retrieve data associated with those accounts. Both requests returned HTTP 200 responses, confirming the server returned

the requested user data without performing any authorization check to verify that the requesting user was entitled to access those specific records.

The two requests were made within one second of each other (12:18:02 and 12:18:03), indicating rapid sequential enumeration of user IDs. In a real-world scenario, this technique — sometimes called IDOR enumeration or horizontal privilege escalation — allows an attacker to iterate through all user_id values and systematically harvest all user records from the application database. Given the prior SQL injection attacks (Attacks 03, 04, 06) against the same environment, the attacker likely already knew the user IDs from the login table enumeration and was directly targeting known accounts.

IDOR Exploitation Details

The following table details the two unauthorized object access events captured:

Timestamp	Parameter	Value	HTTP Status	Impact
May 6, 2025 @ 12:18:02	`user_id`	1	200 OK	First user record (likely admin/root account) exposed
May 6, 2025 @ 12:18:03	`user_id`	3	200 OK	Third user account record exposed

What makes this attack significant — access control analysis:

Expected Behavior (Secure)	Server verifies session user matches requested user_id; returns 403 Forbidden if mismatch
Observed Behavior (Vulnerable)	Server returns HTTP 200 with data for any user_id supplied — no authorization check performed
Attack Scalability	Attacker can iterate user_id=1, 2, 3... N to extract all user records — full database horizontal escalation

Technical Analysis

Indicator	Observed Value / Detail
Elastic Query	event.dataset: apache.access AND url.query > 100 AND http.response.status_code: 200
Vulnerable Parameter	user_id (GET query string parameter — directly referencing database record ID)
Access Control Flaw	No server-side authorization check — any user_id value accepted and returns data
HTTP Method	GET — IDOR exploited via URL query parameter manipulation
Server Response	HTTP 200 OK for both user_id=1 and user_id=3
Attack Speed	Both requests within 1 second (12:18:02 and 12:18:03) — rapid manual

	or scripted enumeration
Context in Attack Chain	Follows SQLi attacks 03, 04, 06 — attacker already knows valid user IDs from login table dump
Horizontal Privilege Escalation	Attacker accessing peer accounts — not admin escalation, but cross-account data theft
Agent Ephemeral ID	d5c32d76-2e83-4b42-b1c7-38b3288d6c61
Ingestion Delay	~18 seconds (event.created 12:18:36 vs event.ingested 12:18:47)

Attack Timeline

Timestamp	Event
May 6, 2025 @ 12:18:02.000	IDOR access — url.query: user_id=1 — HTTP 200 returned — first user record exposed
May 6, 2025 @ 12:18:03.000	IDOR access — url.query: user_id=3 — HTTP 200 returned — third user record exposed
May 6, 2025 @ 12:18:36.270	Events created in Filebeat on host btweb-virtual-machine
May 6, 2025 @ 12:18:47.397	First event ingested into Elastic SIEM (~18 second pipeline delay)
May 6, 2025 @ 12:18:47.398	Second event ingested into Elastic SIEM (1ms after first)

Evidence & Screenshot

Figure 15 — Elastic SIEM: IDOR Events Showing user_id Parameter Manipulation

The Elastic SIEM query filters for Apache access events with url.query values greater than 100 and HTTP 200 responses, returning 2 documents. The url.query field clearly shows user_id=1 and user_id=3 — two different user accounts accessed within one second. Both events share the same agent ephemeral ID (d5c32d76-2e83-4b42-b1c7-38b3288d6c61) and event.created timestamp (12:18:36.270), confirming they originated from the same request session on btweb-virtual-machine.

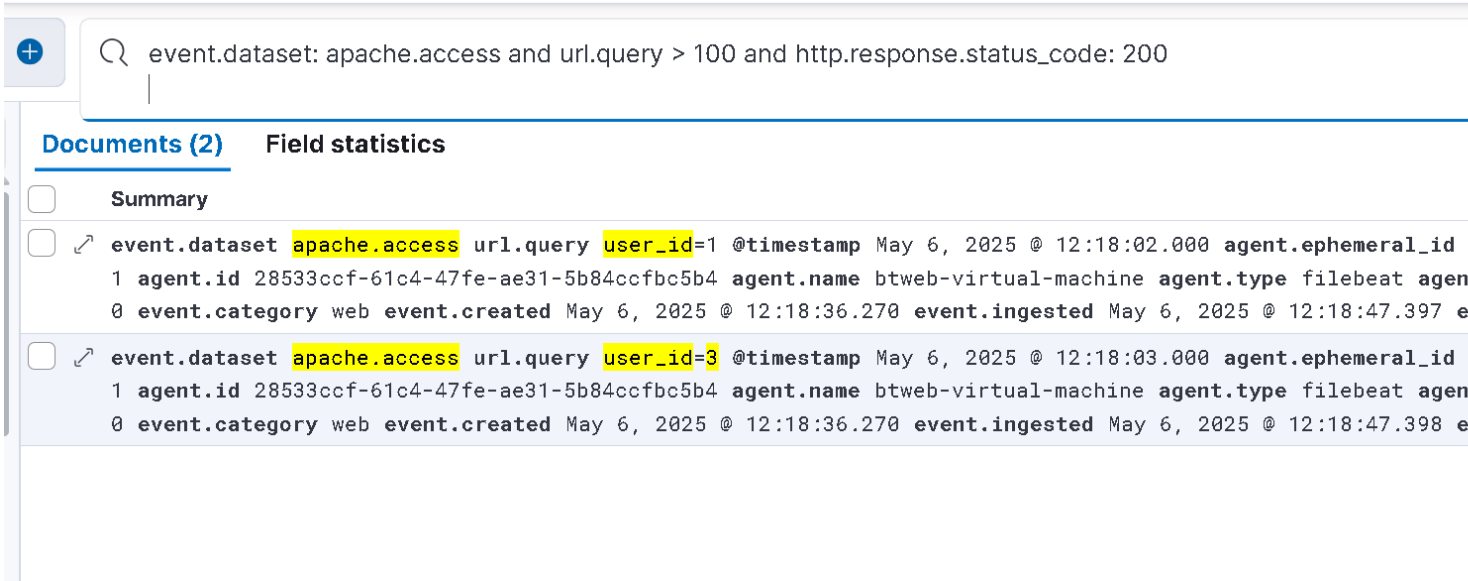


Figure 15: Elastic SIEM — 2 IDOR events showing user_id=1 and user_id=3 accessed with HTTP 200 responses on May 6, 2025

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Defense Evasion	**T1078**	Valid Accounts
Collection	**T1213**	Data from Information Repositories
Collection	**T1005**	Data from Local System
Exfiltration	**T1041**	Exfiltration Over C2 Channel

Recommendations & Remediation

The following controls are essential to prevent Insecure Direct Object Reference attacks:

- Implement server-side authorization checks on every request that accesses a user-specific object — verify that the authenticated session's user ID matches the requested user_id before returning any data.
- Replace direct database IDs (user_id=1, user_id=3) with indirect, non-guessable references such as UUIDs or cryptographic tokens that cannot be easily enumerated.
- Adopt an access control framework or middleware that enforces object-level authorization policies centrally across all endpoints.
- Conduct a full application audit to identify all other endpoints that accept user_id or similar direct object reference parameters and verify each has proper authorization enforcement.
- Configure Elastic SIEM to alert when a single session accesses multiple distinct user_id values within a short time window — a strong indicator of IDOR enumeration.

- Review server logs for the 12:18 window to determine whether additional user IDs beyond 1 and 3 were accessed and what data was returned.
- Implement rate limiting on endpoints accepting user object identifiers to slow down automated enumeration attempts.

ATTACK 08

Investigating File Upload Vulnerability Attack

Attack Overview

Attack ID	SO-WEB-08
Attack Type	Malicious File Upload — Web Shell Deployment & Remote Code Execution
Severity	Critical
Detection Source	Elastic SIEM — Apache Access Logs + Wireshark PCAP
Target Host	btweb-virtual-machine
Target IP	172.16.25.15
Source IP (Attacker)	10.20.10.4
Upload Endpoint	/upload/index.php
Shell Access Endpoint	/upload/index.php?page=uploads/shell.php
Uploaded File	shell.php (web shell)
Shell Execution Response	HTTP 200 OK — text/html (shell responded successfully)
Malicious Extensions Queried	*php, *exe, *msi, *ps1, *vbs
Total Events (Phase 1)	6 documents — malicious file type access attempts
Total Events (Phase 2)	2 documents — confirmed shell.php execution via page parameter
Timestamp (First)	May 7, 2025 @ 13:10:49.000
Timestamp (Shell Exec #1)	May 7, 2025 @ 13:12:17.000
Timestamp (Shell Exec #2)	May 7, 2025 @ 13:12:29.000
Wireshark Shell Frame	Frame 436 — GET /upload/index.php?page=uploads/shell.php — HTTP 200
TCP Reset Observed	Frame 441 — TCP RST from port 4444 — reverse shell connection attempt
Agent Ephemeral ID	60622c48-d496-4e11-a523-0bd16a18e8aa
Agent ID	28533ccf-61c4-47fe-ae31-5b84ccfbc5b4
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0

Attack Description

A File Upload Vulnerability attack occurs when a web application allows users to upload files without adequately validating file type, content, or extension. By uploading a malicious PHP web shell disguised as or directly named shell.php, the attacker places executable server-side code within the web-accessible uploads directory. Once uploaded, the attacker can access the shell via a simple GET request, effectively gaining Remote Code Execution (RCE) on the web server — the most severe possible web application compromise.

This incident unfolded in two distinct phases. In Phase 1, the attacker probed the upload endpoint by requesting URLs containing dangerous file extensions — *php, *exe, *msi, *ps1, and *vbs — generating 6 Apache access events between 13:10:49 and 13:12:29. The Elastic SIEM query detected these attempts by filtering for the malicious extension keywords in url.query. In Phase 2, a refined query filtering specifically for page=uploads/* combined with malicious extensions returned 2 events — the two confirmed executions of the uploaded shell.php file at 13:12:17 and 13:12:29 via the page parameter of the upload portal.

The Wireshark packet capture provides critical additional evidence. The filter http.request.uri == "/upload/index.php?page=uploads/shell.php" reveals two GET requests from 10.20.10.4 to 172.16.25.15, both receiving HTTP 200 OK text/html responses — confirming the shell was successfully executed on the server. Furthermore, a TCP RST packet on port 4444 (Frame 441) is observed immediately after the first shell execution — a strong indicator of a reverse shell callback attempt. Port 4444 is the default listener port for Metasploit and netcat reverse shells, suggesting the uploaded shell attempted to establish a command-and-control connection back to the attacker's machine.

Attack Phases

Phase	Activity	Detection / Evidence	Outcome
Phase 1 — Upload Probe	Attacker requests URLs with .php, .exe, .msi, .ps1, . vbs extensions via upload portal	Elastic: 6 events matching url.query extension filter (13:10:49 – 13:12:29)	**Shell file uploaded to server**
Phase 2 — Shell Execution	Attacker accesses uploaded shell.php via page=uploads/shell.php parameter	Elastic: 2 confirmed events; Wireshark: HTTP 200 response for shell.php	**Remote Code Execution achieved**
Phase 3 — Reverse Shell	Shell attempts to connect back to attacker on port 4444 (Metasploit default)	Wireshark: TCP RST frame 441 on port 4444 — connection reset by server/firewall	**Reverse shell partially blocked**

Technical Analysis

Indicator	Observed Value / Detail
Elastic Query (Phase 1)	event.dataset: apache.access AND url.query.keyword: *php or *exe or *msi or *ps1 or *vbs
Elastic Query (Phase 2)	event.dataset: apache.access AND url.query.keyword: *page=uploads* AND url.query.keyword: *php or *exe or *msi or *ps1 or *vbs

Wireshark Filter (Shell)	http.request.uri == "/upload/index.php?page=uploads/shell.php"
Wireshark Filter (TCP)	ip.src==10.20.10.4
Uploaded Shell File	shell.php — placed in /upload/uploads/ directory via file upload functionality
Shell Access URL	/upload/index.php?page=uploads/shell.php
Shell HTTP Response	200 OK — text/html — 429 bytes — server executed the PHP shell successfully
Reverse Shell Port	4444 — Metasploit meterpreter / netcat default reverse shell callback port
TCP RST on Port 4444	Frame 441 — RST from 10.20.10.4:4444 — reverse shell connection attempt reset
New Source IP	10.20.10.4 (different from prior attacks using 10.20.10.3 — possible second attacker machine or pivot)
Wireshark Frames	Frame 436 (first shell GET), 442 (200 OK), 460 (second GET), 466 (200 OK)
Event Created	May 7, 2025 @ 13:13:39.105 – 13:13:39.106
Event Ingested	May 7, 2025 @ 13:13:49.165 – 13:13:49.172

Attack Timeline

Timestamp	Event
May 7, 2025 @ 13:10:49.000	Phase 1 begins — first malicious extension URL access detected at upload portal
May 7, 2025 @ 13:11:12.000	Second upload probe — continued extension-based file access attempts
May 7, 2025 @ 13:11:22.000	Third probe — attacker testing upload functionality and file access paths
May 7, 2025 @ 13:12:13.000	Fourth probe — upload path mapping in progress
May 7, 2025 @ 13:12:17.000	**FIRST SHELL EXECUTION — page=uploads/shell.php — HTTP 200 OK — RCE confirmed**
May 7, 2025 @ 13:12:29.000	**SECOND SHELL EXECUTION — shell.php accessed again — confirmed persistent access**
Wireshark Frame 436 (t=56.3s)	GET /upload/index.php?page=uploads/shell.php — 441 bytes — from 10.20.10.4
Wireshark Frame 441 (t=56.85s)	TCP RST on port 4444 — reverse shell callback attempt detected and reset
Wireshark Frame 442 (t=57.1s)	HTTP 200 OK — text/html — 429 bytes — shell execution confirmed
Wireshark Frame 460 (t=64.7s)	Second shell GET — shell re-executed for persistent command access
May 7, 2025 @ 13:13:39.105	Events created in Filebeat on host
May 7, 2025 @ 13:13:49.165	Events ingested into Elastic SIEM

Evidence & Screenshots

Figure 16 — Elastic SIEM: 6 Malicious File Extension Access Events (Phase 1)

The Elastic query filtering Apache access logs for malicious extension patterns in url.query returns 6 documents spanning 13:10:49 to 13:12:29. All events share the same agent ephemeral ID (60622c48-d496-4e11-a523-0bd16a18e8aa) and source IP 10.20.10. (truncated in display). The consistent agent confirms all activity originates from a single session on btweb-virtual-machine, representing the attacker's Phase 1 probing of the upload functionality.

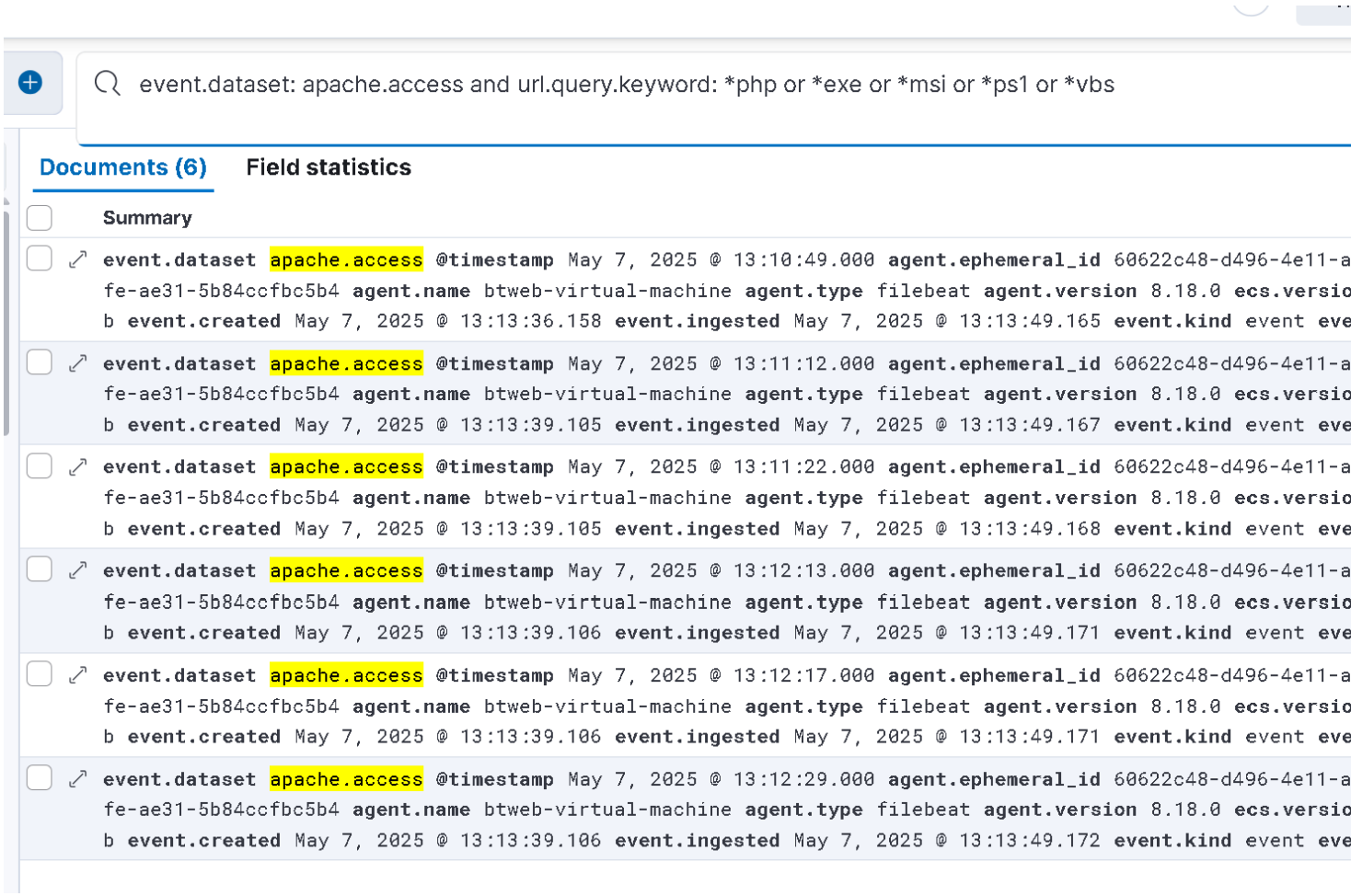


*Figure 16: Elastic SIEM — 6 events with malicious file extension patterns (*php, *exe, *msi, *ps1, *vbs) in url.query on May 7, 2025*

Figure 17 — Elastic SIEM: 2 Confirmed Shell Execution Events (Phase 2)

The refined Elastic query adding url.query.keyword: *page=uploads* narrows results to 2 documents — the two confirmed shell.php executions at 13:12:17 and 13:12:29. These events confirm the attacker successfully uploaded shell.php to the uploads directory and executed it twice through the page parameter of the upload portal.

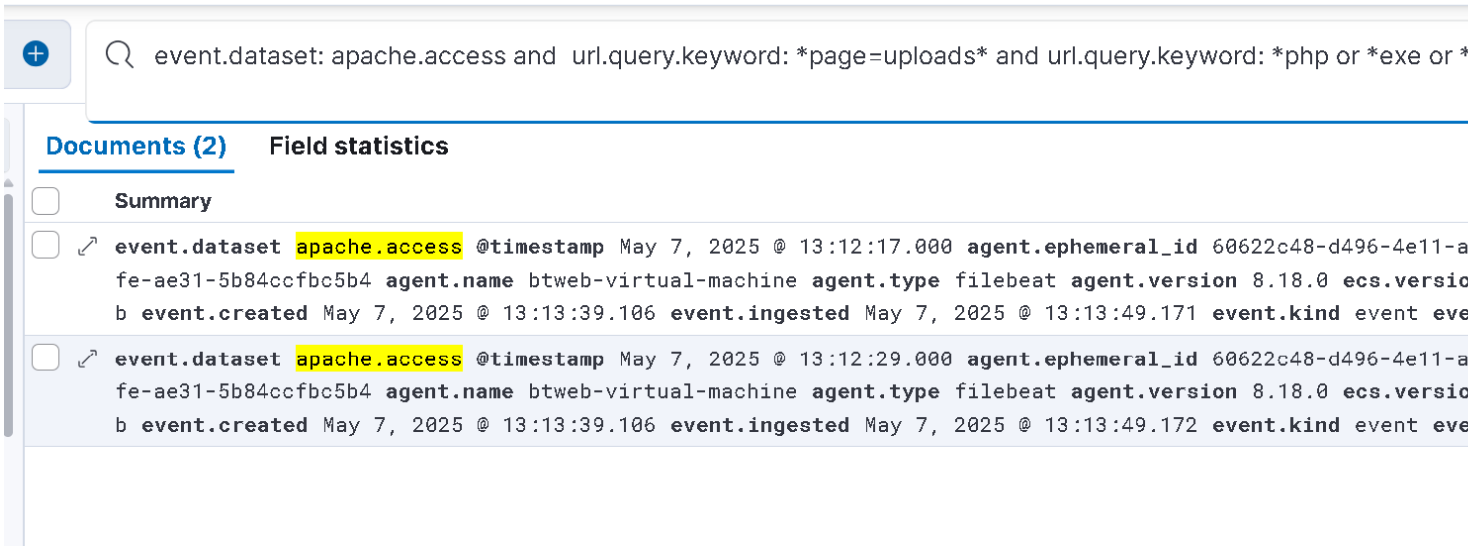


Figure 17: Elastic SIEM — 2 confirmed shell.php execution events via page=uploads/shell.php parameter

Figure 18 — Wireshark: Shell.php GET Requests with HTTP 200 Responses

Wireshark filtered for the exact shell execution URI confirms two GET requests to /upload/index.php?page=uploads/shell.php from 10.20.10.4 to 172.16.25.15 (frames 436 and 460), each receiving HTTP 200 OK (text/html, 429 bytes) responses (frames 442 and 466). This definitively proves the PHP web shell was executed successfully on the server twice.

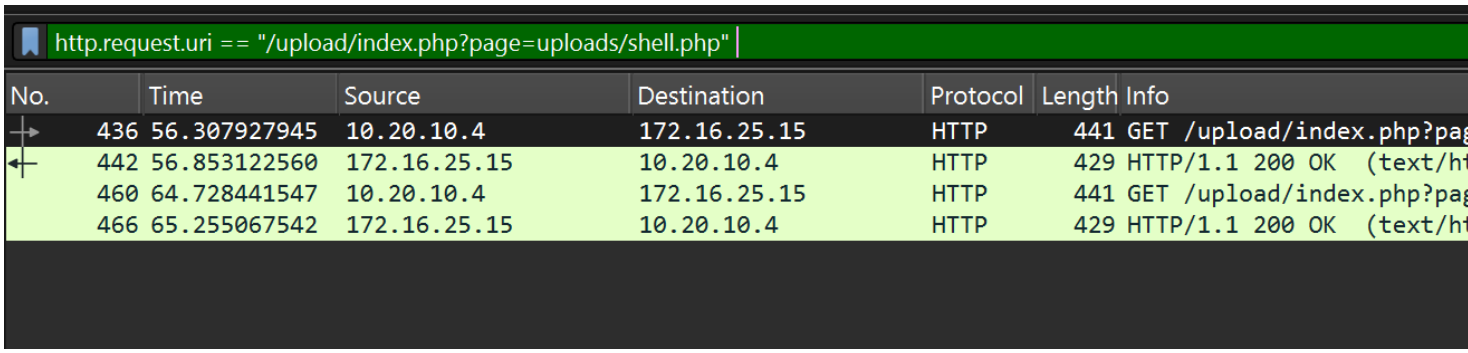


Figure 18: Wireshark — two GET /upload/index.php?page=uploads/shell.php requests with HTTP 200 OK responses confirming RCE

Figure 19 — Wireshark: TCP Activity Showing Port 4444 Reverse Shell Attempt

The broader TCP capture filtered for source IP 10.20.10.4 reveals the full network context around the shell execution. Frame 441 (highlighted in red) shows a TCP RST packet from port 4444 to port 54840 — the server or firewall resetting an inbound connection on port 4444. This is a definitive indicator of a reverse shell callback attempt. The TCP SYN packets on frames 455 and 457 followed by the second shell GET (frame 460) show the attacker re-establishing the HTTP connection after the reverse shell reset, then executing the shell a second time.

ip.src==10.20.10.4

No.	Time	Source	Destination	Protocol	Length	Info
441	56.852151541	10.20.10.4	172.16.25.15	TCP	60	4444 → 54840 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
443	57.108987822	10.20.10.4	172.16.25.15	TCP	66	54436 → 80 [FIN, ACK] Seq=405 Ack=365 Win=31872
445	57.383840900	10.20.10.4	172.16.25.15	TCP	66	54468 → 80 [ACK] Seq=376 Ack=364 Win=31872 Len=0
446	57.384395691	10.20.10.4	172.16.25.15	TCP	66	54452 → 80 [FIN, ACK] Seq=1 Ack=1 Win=32128 Len=
448	57.791821797	10.20.10.4	172.16.25.15	TCP	66	54452 → 80 [ACK] Seq=2 Ack=2 Win=32128 Len=0 TSv
449	61.370843857	10.20.10.4	172.16.25.15	TCP	66	54472 → 80 [FIN, ACK] Seq=1 Ack=1 Win=32128 Len=
451	61.633544702	10.20.10.4	172.16.25.15	TCP	66	54472 → 80 [ACK] Seq=2 Ack=2 Win=32128 Len=0 TSv
453	62.114001015	10.20.10.4	172.16.25.15	TCP	66	54468 → 80 [FIN, ACK] Seq=376 Ack=365 Win=31872
455	64.206004281	10.20.10.4	172.16.25.15	TCP	74	56934 → 80 [SYN] Seq=0 Win=32120 Len=0 MSS=1386
457	64.467615103	10.20.10.4	172.16.25.15	TCP	74	56938 → 80 [SYN] Seq=0 Win=32120 Len=0 MSS=1386
459	64.728441286	10.20.10.4	172.16.25.15	TCP	66	56934 → 80 [ACK] Seq=1 Ack=1 Win=32128 Len=0 TSv
460	64.728441547	10.20.10.4	172.16.25.15	HTTP	441	GET /upload/index.php?page=uploads/shell.php HTT
463	64.990130761	10.20.10.4	172.16.25.15	TCP	66	56938 → 80 [ACK] Seq=1 Ack=1 Win=32128 Len=0 TSv
464	65.252505533	10.20.10.4	172.16.25.15	TCP	74	4444 → 59644 [SYN, ACK] Seq=0 Ack=1 Win=31856 Le
467	65.808202108	10.20.10.4	172.16.25.15	TCP	66	56934 → 80 [ACK] Seq=376 Ack=364 Win=31872 Len=0

Figure 19: Wireshark TCP capture — Frame 441 TCP RST on port 4444 (reverse shell callback attempt) followed by second shell execution

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Execution	**T1059.004**	Command and Scripting Interpreter: Unix Shell
Persistence	**T1505.003**	Server Software Component: Web Shell
Command & Control	**T1071.001**	Application Layer Protocol: Web Protocols
Command & Control	**T1572**	Protocol Tunneling (reverse shell via port 4444)
Exfiltration	**T1041**	Exfiltration Over C2 Channel

Recommendations & Remediation

The following controls are critical to prevent file upload attacks and web shell deployment:

- Implement strict server-side file type validation — only allow explicitly whitelisted MIME types (e.g., image/jpeg, image/png) and reject all executable types (.php, .exe, .msi, .ps1, .vbs, .sh, .py).
- Never store uploaded files in a web-accessible directory that permits script execution — move uploaded files to a location outside the web root, or configure Apache/Nginx to deny PHP execution in upload directories.
- Add an .htaccess rule or server configuration to disable PHP execution in the /upload/uploads/ directory: php_flag engine off.
- Immediately delete or quarantine the shell.php file from the server and audit the uploads directory for any other malicious files uploaded during or before this incident.
- Block outbound connections on port 4444 (and other common reverse shell ports: 1234, 4445, 9001) at the firewall to prevent successful reverse shell callbacks.
- Rename uploaded files to random non-executable filenames and strip file extensions before storage to prevent direct execution.

- Investigate the source IP 10.20.10.4 — this differs from the primary attacker IP (10.20.10.3) and may indicate a second compromised machine, a pivot, or a coordinated multi-system attack.
- Deploy file integrity monitoring on the web server to alert on any new .php files appearing in the uploads or web root directories.

ATTACK 09

Investigating XSS Vulnerability Attack

Attack Overview

Attack ID	SO-WEB-09
Attack Type	Cross-Site Scripting (XSS) — Reflected / Stored XSS Attempt
Severity	High
Detection Source	Elastic SIEM — Apache Access Logs
Target Host	btweb-virtual-machine
XSS Patterns Detected	*prompt*, *onclick*, *script*, *javascript*, *onFocus*
Total Events	2 matching documents
Timestamp (First)	May 7, 2025 @ 18:22:43.000
Timestamp (Second)	May 7, 2025 @ 18:22:57.000
Time Between Events	14 seconds
Event Created	May 7, 2025 @ 18:25:03.923
Event Ingested	May 7, 2025 @ 18:25:15.027 / 18:25:15.031
Agent Ephemeral ID	83ce8940-c321-4e26-8c80-0c504f04ba15
Agent ID	28533ccf-61c4-47fe-ae31-5b84ccfbc5b4
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0
ECS Version	1.12.0
Event Category	web

Attack Description

Cross-Site Scripting (XSS) is a client-side injection attack that allows an adversary to inject malicious scripts into web pages viewed by other users. Unlike server-side attacks (SQLi, RCE), XSS targets the victim's browser — not the server — enabling the attacker to steal session cookies, capture credentials, redirect users to malicious sites, perform actions on behalf of authenticated users, or deliver malware. XSS is consistently ranked in the OWASP Top 10 (A03: Injection) and remains one of the most prevalent web vulnerabilities.

In this incident, the attacker injected XSS payloads into URL query parameters on the target web application. The Elastic SIEM query detected two events containing known XSS indicator patterns — prompt(), onclick, <script>, javascript:, and onFocus — in the url.query field of the Apache access logs. Both events occurred within 14 seconds of each other (18:22:43 and 18:22:57), indicating the attacker was testing multiple XSS payload variants in rapid succession.

The detected XSS keyword patterns correspond to the most commonly used JavaScript injection techniques: prompt() is a JavaScript function used in XSS proof-of-concept payloads to demonstrate script execution by triggering a browser dialog. onclick and onFocus are HTML event handlers that execute JavaScript when a user interacts with an element. The <script> tag is the most direct XSS vector. javascript: is a URI scheme used to execute JavaScript inline. Together, these patterns indicate a structured XSS discovery probe testing both reflected and DOM-based injection vectors across the application's input fields.

XSS Payload Patterns Detected

The following XSS indicator patterns were included in the Elastic detection query:

XSS Pattern	Example Payload	Attack Vector / Purpose
`*prompt*`	`<script>prompt(1)</script>`	JavaScript dialog PoC — confirms script execution in browser context
`*onclick*`	`<img src=x onclick=alert(1)>`	HTML event handler injection — fires on user interaction
`*script*`	`<script>maliciousCode()</script>`	Direct script tag injection — most common XSS vector
`*javascript*`	`<a href=javascript:alert(1)>Click</a>`	JavaScript URI scheme — executes on link click
`*onFocus*`	`<input autofocus onfocus=alert(1)>`	Auto-trigger event handler — fires without user interaction if autofocus used

XSS Attack Types — Context

XSS Type	How It Works	Potential Impact
Reflected XSS	Payload injected via URL parameter is reflected in the server response and executed in victim's browser immediately	Session hijacking, credential theft, phishing redirection
Stored XSS	Payload is saved to the database and served to all users who view the affected page	Persistent compromise, worm propagation, mass credential theft
DOM-Based XSS	Payload modifies the DOM client-side without server involvement — triggered by JavaScript processing URL fragments	Bypasses server-side WAF filters, session token theft

Technical Analysis

Indicator	Observed Value / Detail
Elastic Query	event.dataset: apache.access AND url.query.keyword: (*prompt* OR *onclick* OR *script* OR *javascript* OR *onFocus*)

Detection Method	XSS keyword pattern matching in url.query field of Apache access logs
Total Events	2 documents — two distinct XSS payload injections 14 seconds apart
Injection Vector	URL query parameter — GET request with XSS payload embedded in query string
Attack Speed	14-second interval (18:22:43 and 18:22:57) — rapid manual or tool-assisted probing
Payload Variations	Multiple XSS pattern types tested — indicates systematic XSS discovery methodology
Source IP (Partial)	10.20.10. (truncated in display) — same subnet as prior attacks
Agent Ephemeral ID	83ce8940-c321-4e26-8c80-0c504f04ba15
Event Created	May 7, 2025 @ 18:25:03.923 (same for both events — batch creation)
Ingestion Delay	~2 minutes 22 seconds (18:22:57 attack vs 18:25:15 ingested)
Same Attack Chain	Same agent.id (28533ccf) as all previous attacks — confirms single attacker session

Attack Timeline

Timestamp	Event
May 7, 2025 @ 18:22:43.000	First XSS payload injected — url.query containing *prompt*, *onclick*, *script*, *javascript*, or *onFocus* pattern detected
May 7, 2025 @ 18:22:57.000	Second XSS payload injected — 14-second interval confirms attacker testing multiple payload variants
May 7, 2025 @ 18:25:03.923	Both events created in Filebeat on btweb-virtual-machine (batch creation — same timestamp)
May 7, 2025 @ 18:25:15.027	First event ingested into Elastic SIEM (~2 min 22 sec pipeline delay)
May 7, 2025 @ 18:25:15.031	Second event ingested into Elastic SIEM (4ms after first)

Evidence & Screenshot

Figure 20 — Elastic SIEM: XSS Payload Patterns Detected in Apache Access Logs

The Elastic SIEM query filtering Apache access logs for XSS indicator keywords in url.query.keyword returns 2 documents. Both events share the same agent ephemeral ID (83ce8940-c321-4e26-8c80-0c504f04ba15) and the same event.created timestamp (18:25:03.923), confirming they were generated in the same Filebeat collection cycle. The 14-second gap between the two attack timestamps (18:22:43 and 18:22:57) indicates a deliberate, structured probe testing multiple XSS injection vectors against the application's URL query parameters.

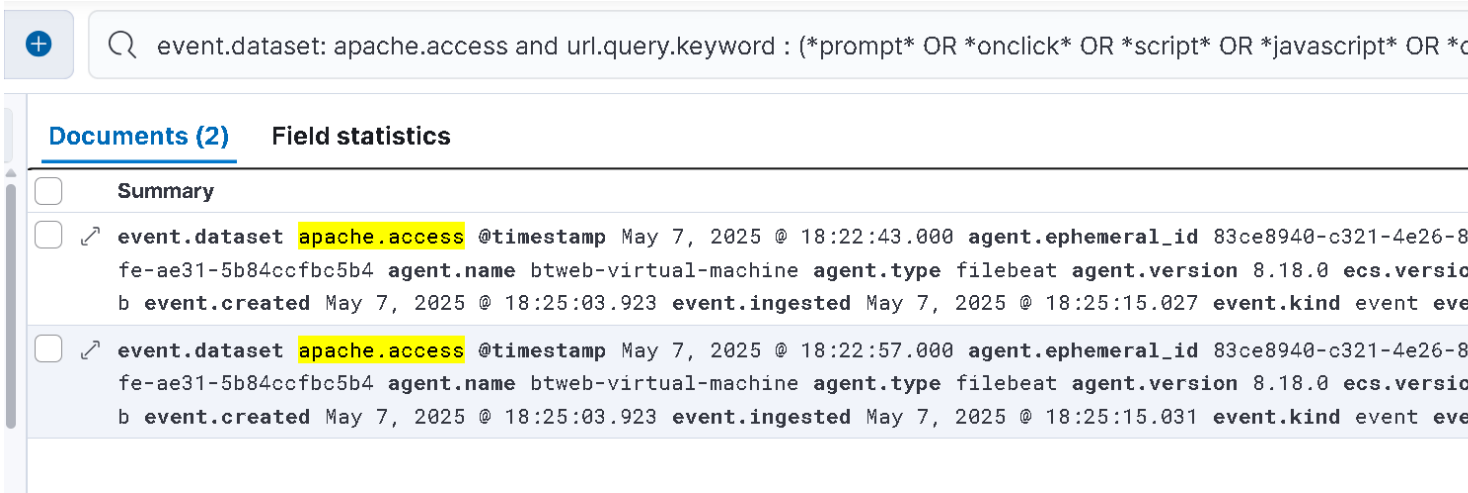


Figure 20: Elastic SIEM — 2 XSS events with prompt/onclick/script/javascript/onFocus patterns in url.query on May 7, 2025

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Execution	**T1059.007**	Command and Scripting Interpreter: JavaScript
Credential Access	**T1539**	Steal Web Session Cookie
Collection	**T1185**	Browser Session Hijacking
Defense Evasion	**T1027**	Obfuscated Files or Information

Recommendations & Remediation

The following controls are essential to prevent Cross-Site Scripting attacks:

- Implement strict output encoding for all user-supplied data rendered in HTML responses — encode <, >, ', ", & and JavaScript special characters before reflection in the page.
- Apply a Content Security Policy (CSP) header to restrict which scripts can execute — use script-src 'self' to block inline scripts and unauthorized external script sources.
- Validate and sanitize all input server-side — reject or strip HTML tags, JavaScript event handlers, and URI scheme patterns (javascript:) from query parameters and form fields.
- Set the HttpOnly flag on all session cookies to prevent JavaScript from accessing them — this defeats session hijacking as the primary XSS objective.
- Set the Secure flag on all cookies to ensure they are only transmitted over HTTPS connections.
- Deploy a WAF with XSS detection rules to identify and block payloads containing <script>, javascript:, and event handler patterns (onclick, onfocus, onerror) in HTTP parameters.
- Expand the Elastic SIEM XSS detection query to include additional patterns: *onerror*, *onload*, *eval(*, *document.cookie*, *fetch(* — to broaden coverage of advanced XSS techniques.

- Conduct a full application XSS audit — test all input fields, URL parameters, and HTTP headers for reflection vulnerabilities using a tool such as Burp Suite or OWASP ZAP.

ATTACK 10

Investigating SSRF (Server-Side Request Forgery)

Attack Overview

Attack ID	SO-WEB-10
Attack Type	Server-Side Request Forgery (SSRF) — Internal Network Probing
Severity	Critical
Detection Source	Wireshark PCAP — HTTP Traffic Analysis
Target Host	172.16.25.15 (web server — used as SSRF pivot)
Source IP (Attacker)	10.20.10.3
Vulnerable Endpoint	/fetch
Injection Parameter	url (GET query parameter)
SSRF Probe Sequence	Hello → google.com → test.com → http://127.0.0.1:5000/internal → http://127.0.0.1:8080/secret.txt
Final Target (Internal)	http://127.0.0.1:8080/secret.txt
Decoded SSRF URL	http://127.0.0.1:8080/secret.txt (via CyberChef URL Decode)
External Outbound IPs	185.125.190.49, 185.125.190.98, 91.189.91.98 (server making outbound requests)
HTTP Method	GET
HTTP Version	HTTP/1.1
Wireshark Filter	http && http.request.method == "GET"

Attack Description

Server-Side Request Forgery (SSRF) is a critical web vulnerability that allows an attacker to induce the server to make HTTP requests to arbitrary URLs — including internal services, cloud metadata endpoints, and loopback addresses — that would normally be inaccessible from the external network. SSRF effectively turns the vulnerable web server into an unwitting proxy, allowing the attacker to access internal infrastructure, bypass firewall rules, and enumerate internal services from behind the perimeter. It is ranked in the OWASP Top 10 (A10: SSRF) and has been used in major cloud breaches to access instance metadata services.

In this incident, the attacker exploited the /fetch endpoint which accepts a url GET parameter and makes server-side HTTP requests to the supplied URL. The attack followed a methodical escalation pattern: starting with benign probes (url=Hello, url=google.com, url=test.com) to confirm the server was fetching external URLs, then pivoting to internal loopback addresses. First targeting http://127.0.0.1:5000/internal to probe an internal service on port 5000, then escalating to http://127.0.0.1:8080/secret.txt — attempting to read a sensitive file from an internal service running

on port 8080. This URL was submitted three times, confirming the attacker was persistently attempting to extract the secret.txt content.

The Wireshark capture also reveals the server making outbound HTTP requests to external IPs (185.125.190.49, 185.125.190.98, 91.189.91.98) — these are the server's own outbound requests triggered by the SSRF payloads for google.com and test.com, confirming the vulnerability is active and the server is faithfully fetching any URL provided. The URL-encoded SSRF payload was decoded using CyberChef, confirming the final target as http://127.0.0.1:8080/secret.txt — a direct attempt to exfiltrate internal application secrets.

SSRF Payload Escalation Sequence

The attacker followed a deliberate step-by-step escalation from external to internal SSRF:

Step	Wireshark Frame	Payload Sent	Purpose
1	Frame 4	`GET / HTTP/1.1`	Initial probe — testing connectivity to web server
2	Frame 18	`GET /fetch?url=Hello`	Testing if /fetch endpoint processes arbitrary input
3	Frame 32	`GET /fetch?url=google.com`	External URL test — confirms server makes outbound HTTP requests
4	Frame 68	`GET /fetch?url=test.com`	Second external probe — validates SSRF via different domain
5	Frame 95	`GET /fetch?url=http%3A%2F%2F127.0.0.1%3A5000%2Finternal`	Internal probe — 127.0.0.1:5000/internal — first loopback escalation
6	Frame 132	`GET /fetch?url=http%3A%2F%2F127.0.0.1%3A5000%2Finternal`	Repeated internal probe on port 5000 — confirming service presence
7	Frame 167	`GET /fetch?url=http%3A%2F%2F127.0.0.1%3A8080%2Fsecret.txt`	**TARGET: 127.0.0.1:8080/secret.txt — sensitive file exfiltration attempt**
8	Frame 186	`GET /fetch?url=http%3A%2F%2F127.0.0.1%3A8080%2Fsecret.txt`	**Repeated exfiltration attempt — persistent extraction of secret.txt**
9	Frame 200	`GET /fetch?url=http%3A%2F%2F127.0.0.1%3A8080%2Fsecret.txt`	**Third attempt — attacker persistently targeting internal secret file**

Technical Analysis

Indicator	Observed Value / Detail
Vulnerable Endpoint	/fetch — accepts user-supplied url parameter and makes server-side HTTP requests
SSRF Injection Parameter	url (GET query string parameter — no validation or allowlist enforced)
External SSRF Confirmed	Server made outbound requests to 185.125.190.49, .98, 91.189.91.98 (google.com, test.com resolved)
Internal SSRF Target 1	http://127.0.0.1:5000/internal — internal application service probed

Internal SSRF Target 2	http://127.0.0.1:8080/secret.txt — sensitive file on internal port 8080 (3 attempts)
URL Encoding Used	%3A = :, %2F = /, %2F = / — standard URL encoding for SSRF payload obfuscation
Decoded Target URL	http://127.0.0.1:8080/secret.txt (confirmed via CyberChef URL Decode)
Attack Escalation Pattern	External → Internal loopback (port 5000) → Internal secret file (port 8080) — systematic SSRF chain
Repeat Attempts	secret.txt requested 3 times (frames 167, 186, 200) — persistent targeting
Server Outbound IPs	185.125.190.49, 185.125.190.98 (Ubuntu CDN), 91.189.91.98 — fetched by server on attacker's behalf
Wireshark Filter	http && http.request.method == "GET"
Source IP	10.20.10.3 — same primary attacker as Attacks 01-07
Protocol	HTTP/1.1 — unencrypted — all traffic visible in capture

Attack Timeline

Wireshark Time (seconds)	Event
t = 0.247s (Frame 4)	GET / HTTP/1.1 — initial connection to web server established
t = 5.095s (Frame 18)	GET /fetch?url=Hello — SSRF endpoint discovered, benign probe sent
t = 13.635s (Frame 32)	GET /fetch?url=google.com — external URL fetch confirmed; server made outbound request to 185.125.190.49
t = 31.515s (Frame 68)	GET /fetch?url=test.com — second external probe; server fetched 185.125.190.98 and 91.189.91.98
t = 91.092s (Frame 95)	**SSRF PIVOTS INTERNAL — GET /fetch?url=http://127.0.0.1:5000/internal — loopback probe begins**
t = 139.662s (Frame 132)	**Repeated internal probe on 127.0.0.1:5000/internal — confirming service availability**
t = 150.511s (Frame 167)	**CRITICAL: GET /fetch?url=http://127.0.0.1:8080/secret.txt — sensitive file exfiltration attempt #1**
t = 159.467s (Frame 186)	**Repeated secret.txt fetch — exfiltration attempt #2 — 8.9 seconds after first**
t = 164.052s (Frame 200)	**Repeated secret.txt fetch — exfiltration attempt #3 — persistent targeting confirmed**

Evidence & Screenshots

Figure 21 — Wireshark: Full SSRF Escalation Sequence GET Requests

The Wireshark capture filtered for HTTP GET requests shows the complete SSRF attack progression. Starting with /fetch?url=Hello and escalating through google.com, test.com, 127.0.0.1:5000/internal, and finally three repeated requests to 127.0.0.1:8080/secret.txt. The server

outbound requests to external IPs (185.125.190.49, 185.125.190.98, 91.189.91.98) confirm the SSRF is active and the server is faithfully proxying requests to attacker-controlled URLs.

http && http.request.method == "GET"

No.	Time	Source	Destination	Protocol	Length	Info
4	0.247058017	10.20.10.3	172.16.25.15	HTTP	488	GET / HTTP/1.1
18	5.095191813	10.20.10.3	172.16.25.15	HTTP	541	GET /fetch?url=Hello HTTP/1.
32	13.635088925	10.20.10.3	172.16.25.15	HTTP	546	GET /fetch?url=google.com HT
53	27.797538131	172.16.25.15	185.125.190.49	HTTP	153	GET / HTTP/1.1
54	27.800785679	172.16.25.102	185.125.190.98	HTTP	153	GET / HTTP/1.1
68	31.515718677	10.20.10.3	172.16.25.15	HTTP	544	GET /fetch?url=test.com HTTP
84	88.602590893	172.16.25.10	185.125.190.49	HTTP	154	GET / HTTP/1.1
95	91.092095307	10.20.10.3	172.16.25.15	HTTP	576	GET /fetch?url=http%3A%2F%2F
120	134.264981773	172.16.25.5	91.189.91.98	HTTP	154	GET / HTTP/1.1
132	139.662135998	10.20.10.3	172.16.25.15	HTTP	576	GET /fetch?url=http%3A%2F%2F
167	150.511127966	10.20.10.3	172.16.25.15	HTTP	578	GET /fetch?url=http%3A%2F%2F
186	159.467357950	10.20.10.3	172.16.25.15	HTTP	540	GET /fetch?url=http%3A%2F%2F
200	164.052360673	10.20.10.3	172.16.25.15	HTTP	540	GET /fetch?url=http%3A%2F%2F

Figure 21: Wireshark — SSRF escalation sequence from external probes to internal 127.0.0.1:8080/secret.txt (3 repeated attempts)

Figure 22 — Wireshark HTTP Detail: SSRF Request URI Inspection

The Wireshark HTTP protocol dissector for the SSRF request reveals the full decoded URI structure. The Request URI Path is /fetch and the Request URI Query parameter is url=http%3A%2F%2F127.0.0.1%3A8080%2Fsecret.txt — the URL-encoded SSRF payload targeting the internal service. This confirms the injection point, the encoding method used, and the exact internal target at the packet level.

```
▼ Hypertext Transfer Protocol
  ▼ GET /fetch?url=http%3A%2F%2F127.0.0.1%3A8080%2Fsecret.txt HTTP/1.1\r\n
      Request Method: GET
    ▼ Request URI: /fetch?url=http%3A%2F%2F127.0.0.1%3A8080%2Fsecret.txt
        Request URI Path: /fetch
      ▼ Request URI Query: url=http%3A%2F%2F127.0.0.1%3A8080%2Fsecret.txt
          Request URI Query Parameter: url=http%3A%2F%2F127.0.0.1%3A8080%
      Request Version: HTTP/1.1
```

Figure 22: Wireshark HTTP detail — /fetch endpoint with url=http%3A%2F%2F127.0.0.1%3A8080%2Fsecret.txt SSRF payload in query parameter

Figure 23 — CyberChef: URL-Decoded SSRF Target Confirmed

CyberChef URL Decode applied to the raw query parameter confirms the decoded SSRF target: url=http://127.0.0.1:8080/secret.txt. This removes any ambiguity about the attacker's intent — the server was being instructed to make a GET request to the internal loopback address on port 8080 and retrieve the contents of secret.txt, which likely contains application secrets, API keys, or configuration credentials.

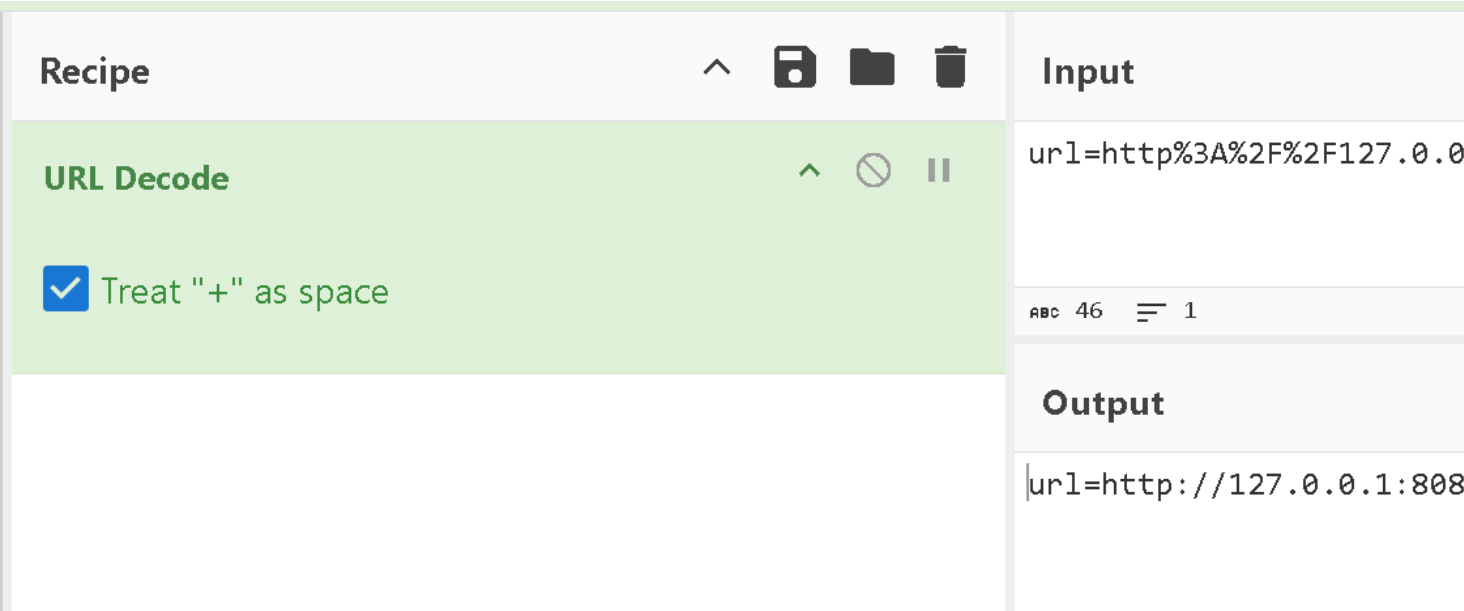


Figure 23: CyberChef URL Decode — confirming SSRF target as http://127.0.0.1:8080/secret.txt

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Discovery	**T1046**	Network Service Discovery (internal port scanning)
Lateral Movement	**T1599**	Network Boundary Bridging
Collection	**T1005**	Data from Local System
Exfiltration	**T1041**	Exfiltration Over C2 Channel
Defense Evasion	**T1090**	Proxy (using server as SSRF proxy)

Recommendations & Remediation

The following controls are essential to prevent Server-Side Request Forgery attacks:

- Implement a strict allowlist for the url parameter — only permit requests to explicitly approved external domains; deny all loopback (127.0.0.1), private RFC-1918, and metadata service addresses.
- Validate and sanitize the url parameter server-side — reject URLs containing 127.0.0.1, localhost, 169.254.169.254 (cloud metadata), and private IP ranges (10.x, 172.16.x, 192.168.x).
- Immediately investigate whether secret.txt was successfully retrieved — check the HTTP response bodies for the three 127.0.0.1:8080/secret.txt requests and rotate any credentials or keys it may contain.

- Disable or redesign the /fetch endpoint — if server-side URL fetching is required, use a dedicated, sandboxed microservice with strict egress filtering, not a general-purpose fetch proxy.
- Apply network-level egress filtering on the web server to block outbound requests to loopback and internal addresses at the OS or firewall level.
- Disable or restrict the internal service on port 8080 — if it serves sensitive files (secret.txt), it must require authentication and not be reachable via SSRF proxy.
- Add Elastic SIEM alerting for Apache access logs containing url=http*127.0.0.1* or url=*localhost* patterns to detect future SSRF attempts targeting internal services.
- Audit all other endpoints that accept URL parameters and apply the same SSRF mitigations — SSRF vulnerabilities are often present in multiple locations within the same application.

ATTACK 11

Investigating Command Injection

Attack Overview

Attack ID	SO-WEB-11
Attack Type	OS Command Injection — Remote Code Execution via Web Form
Severity	Critical
Detection Source	Elastic SIEM — Apache Access Logs + Wireshark PCAP
Target Host	btweb-virtual-machine
Target IP	172.16.25.15
Source IP	10.20.10.3
Target Endpoint	/ (root — likely a ping/network diagnostic form)
HTTP Method	POST
Content-Type	application/x-www-form-urlencoded
Vulnerable Parameter	ip
Injected Command	127.0.0.1; ls -la /root
Injection Delimiter	; (semicolon — chains OS commands)
OS Command Injected	ls -la /root — lists all files in the root user's home directory
Total Events (Elastic)	7 matching POST documents
Timestamp (First)	May 20, 2025 @ 15:58:11.000
Timestamp (Last)	May 20, 2025 @ 16:00:13.000
Wireshark Frames	Frame 8 (t=0.29s), 34 (t=12.23s), 48 (t=23.02s), 68 (t=62.45s)
Agent Ephemeral ID	367f9f1a-9f90-4763-95bb-6257117dd8cf
Agent ID	28533ccf-61c4-47fe-ae31-5b84ccfbc5b4
Agent Name	btweb-virtual-machine
Agent Type / Version	Filebeat 8.18.0
ECS Version	1.12.0

Attack Description

OS Command Injection is one of the most severe web application vulnerabilities, allowing an attacker to execute arbitrary operating system commands on the server hosting the application. It occurs when user-supplied input is passed unsanitized to a system shell function (such as exec(), system(), or shell_exec() in PHP). By injecting shell metacharacters — such as semicolons (;), pipes (|), ampersands (&), or backticks (`) — the attacker can chain additional OS commands after

the intended input, gaining full Remote Code Execution (RCE) on the server with the privileges of the web server process.

In this incident, the attacker targeted a web form at the root endpoint (/) that accepts an ip parameter — characteristic of a network diagnostic tool such as a ping utility exposed via the web interface. Instead of supplying a valid IP address, the attacker injected the payload 127.0.0.1; ls -la /root — using a semicolon to terminate the ping command and chain the OS command ls -la /root. This command lists all files and directories in the Linux root user's home directory (/root), including hidden files (due to the -a flag), along with permissions, ownership, and timestamps (due to -la). This is a classic first-stage command injection reconnaissance payload — confirming RCE and enumerating the root user's file system.

The Elastic SIEM detected 7 POST events to the root endpoint on May 20, 2025, spanning approximately 2 minutes (15:58:11 to 16:00:13). The Wireshark capture confirms 4 raw POST requests from 10.20.10.3 to 172.16.25.15:80, all with application/x-www-form-urlencoded content type. The HTML Form URL Encoded dissector in Wireshark directly reveals the injected form value: ip = 127.0.0.1; ls -la /root — providing unambiguous packet-level evidence of the command injection payload. This is the final and most severe attack in the web attack chain, representing a complete server compromise via OS command execution.

Command Injection Payload Analysis

The injected payload was extracted directly from Wireshark's HTML form dissector:

Field	Submitted Value	Effect / Impact
ip	`127.0.0.1; ls -la /root`	Semicolon chains ping to ls -la /root — lists root home directory contents with full permissions and hidden files

How the injection works at the OS shell level:

Intended Command	`ping -c 4 [ip_input]` → `ping -c 4 127.0.0.1`
Injected Command	`ping -c 4 127.0.0.1; ls -la /root`
Execution Result	Ping executes normally, THEN ls -la /root executes — server returns full directory listing of /root to attacker
Further Escalation	Attacker can substitute any OS command: cat /etc/shadow, wget [url] -O shell.php, id, whoami, chmod, useradd etc.

Technical Analysis

Indicator	Observed Value / Detail
Elastic Query	event.dataset: apache.access AND http.request.method.keyword: POST
Total Elastic Events	7 POST request events detected on May 20, 2025

Wireshark Filter (POST)	http && http.request.method == "POST"
Wireshark Filter (Form)	http && http.request.method == "POST" && urlencoded-form
Vulnerable Parameter	ip — accepts IP address input, passed directly to OS ping function without sanitization
Injection Technique	Semicolon (;) command chaining — most common OS command injection delimiter
Injected OS Command	ls -la /root — directory listing of root user's home directory with hidden files
Command Injection Implication	Any OS command can be executed — whoami, id, cat /etc/shadow, wget malware, reverse shell
Attack Pattern	7 POST requests over ~2 minutes — multiple payloads tested (varying commands likely)
Wireshark Frames	Frames 8, 34, 48, 68 — 4 POST requests captured in PCAP
Source/Dest	10.20.10.3 → 172.16.25.15:80 — same primary attacker as attacks 01-07, 09-10
Content-Type	application/x-www-form-urlencoded — standard HTML form submission
Event Created	May 20, 2025 @ 18:08:28.035 – 18:08:28.044 (batch ingestion ~2hr after attack)
Ingestion Delay	~2 hours 9 minutes (attack 15:58 vs ingestion 18:08) — notable pipeline delay

Attack Timeline

Timestamp	Event
May 20, 2025 @ 15:58:11.000	First POST detected — command injection sequence begins at root endpoint
May 20, 2025 @ 15:58:34.000	Second POST — continued injection attempts (23 second interval)
May 20, 2025 @ 15:58:51.000	Third POST — payload variations being tested
May 20, 2025 @ 15:59:32.000	Fourth POST — 41 second gap — attacker reviewing server responses
May 20, 2025 @ 15:59:47.000	Fifth POST — 15 second interval — refined payload delivery
May 20, 2025 @ 16:00:09.000	Sixth POST — continued exploitation
May 20, 2025 @ 16:00:13.000	Seventh POST — final captured injection event
Wireshark t=0.29s (Frame 8)	First raw POST captured — ip=127.0.0.1; ls -la /root payload submitted
Wireshark t=12.23s (Frame 34)	Second POST — form data confirms command injection payload pattern
Wireshark t=23.02s (Frame 48)	Third POST — consistent application/x-www-form-urlencoded delivery
Wireshark t=62.45s (Frame 68)	Fourth POST — final captured frame — ~62 seconds into the session
May 20, 2025 @ 18:08:28.035	Events created in Filebeat (~2 hours 9 minutes after attack)
May 20, 2025 @ 18:08:39.932	All events ingested into Elastic SIEM

Evidence & Screenshots

Figure 24 — Elastic SIEM: 7 POST Request Events at Root Endpoint

The Elastic SIEM query for POST requests in Apache access logs returns 7 documents on May 20, 2025. All events share the same agent ephemeral ID (367f9f1a-9f90-4763-95bb-6257117dd8cf) and the same agent.id (28533ccf — consistent throughout the entire attack chain). Timestamps span from 15:58:11 to 16:00:13, covering the ~2-minute command injection session. The notable 2-hour gap between attack timestamps and event.created (18:08:28) indicates a delayed Filebeat collection cycle.

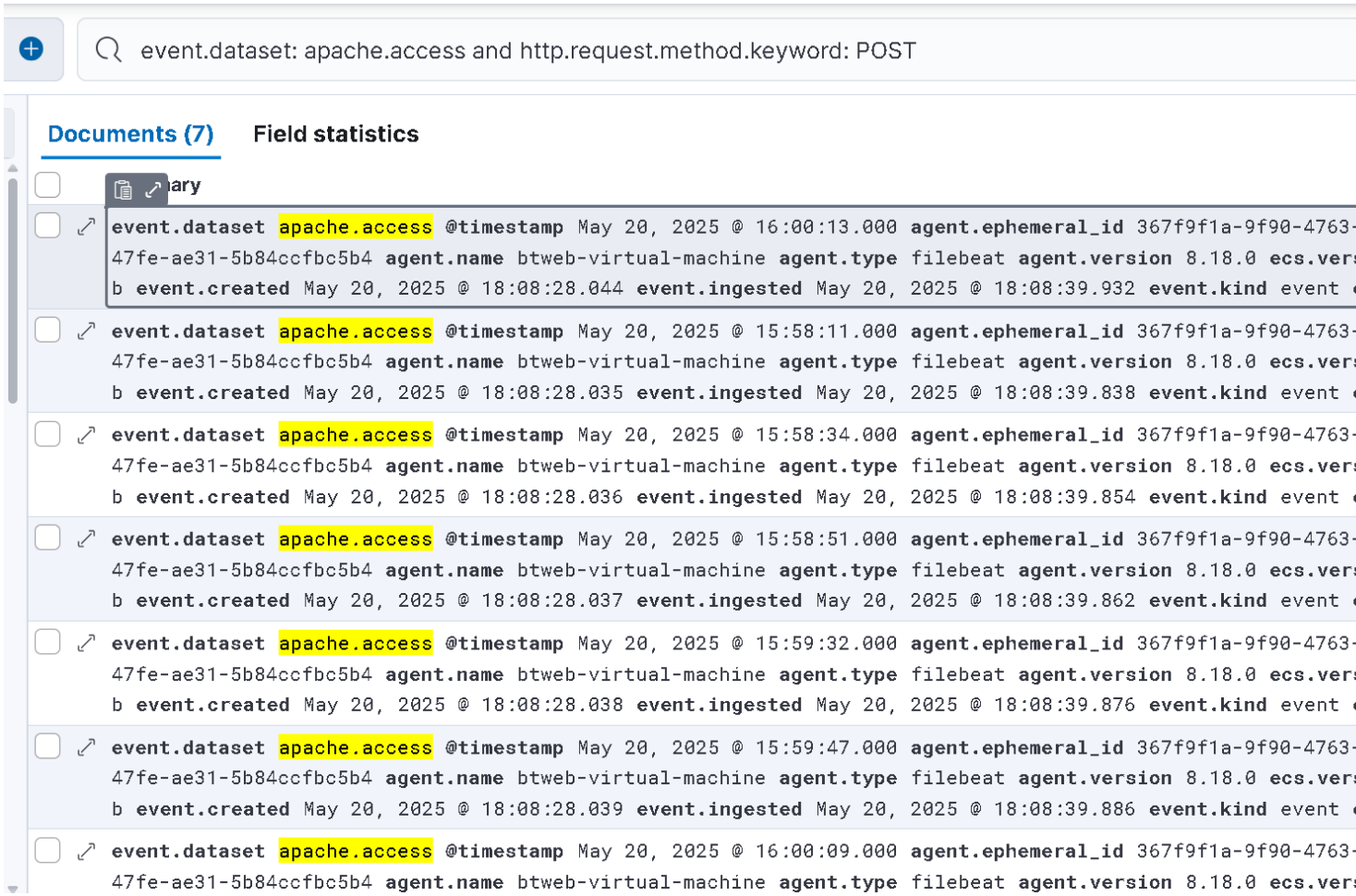


Figure 24: Elastic SIEM — 7 POST events at root endpoint on May 20, 2025, spanning 15:58:11 to 16:00:13

Figure 25 — Wireshark: 4 POST Requests Captured (http && POST filter)

The Wireshark capture filtered for HTTP POST requests shows 4 raw form submissions from 10.20.10.3 to 172.16.25.15:80. All frames show POST / HTTP/1.1 with application/x-www-form-urlencoded content type, spanning from t=0.29s to t=62.45s. Frame 8 is selected (647 bytes), confirming the form data payload is embedded in the request body.

http && http.request.method == "POST"

No.	Time	Source	Destination	Protocol	Length	Info
8	0.289909438	10.20.10.3	172.16.25.15	HTTP	647	POST / HTTP/1.1 (ap
34	12.230294757	10.20.10.3	172.16.25.15	HTTP	667	POST / HTTP/1.1 (ap
48	23.021421145	10.20.10.3	172.16.25.15	HTTP	661	POST / HTTP/1.1 (ap
68	62.449748595	10.20.10.3	172.16.25.15	HTTP	674	POST / HTTP/1.1 (ap

Figure 25: Wireshark — 4 POST requests from 10.20.10.3 to 172.16.25.15 with application/x-www-form-urlencoded command injection payloads

Figure 26 — Wireshark: URL-Encoded Form Filter Confirming POST Body

The refined Wireshark filter adding urlencoded-form to the POST filter confirms all 4 captured requests contain URL-encoded form data in their body. This validates that the ip parameter and its injected value were transmitted as standard HTML form POST data, matching the Apache access log records in Elastic SIEM.

http && http.request.method == "POST" && urlencoded-form

No.	Time	Source	Destination	Protocol	Length	Info
8	0.289909438	10.20.10.3	172.16.25.15	HTTP	647	POST / HTTP/1.1 (appl
34	12.230294757	10.20.10.3	172.16.25.15	HTTP	667	POST / HTTP/1.1 (appl
48	23.021421145	10.20.10.3	172.16.25.15	HTTP	661	POST / HTTP/1.1 (appl
68	62.449748595	10.20.10.3	172.16.25.15	HTTP	674	POST / HTTP/1.1 (appl

Figure 26: Wireshark — urlencoded-form filter confirming all 4 POST requests carry form body data containing the command injection payload

Figure 27 — Wireshark Form Dissector: Command Injection Payload Exposed

The Wireshark HTML Form URL Encoded dissector provides the most direct and definitive evidence of the attack. The form field ip contains the value 127.0.0.1; ls -la /root — the exact command injection payload. This is packet-level proof of OS command injection, captured at the network layer, showing the semicolon-chained ls -la /root command injected into the ip parameter of the web form.

```
▾ HTML Form URL Encoded: application/x-www-form-urlen
  ▾ Form item: "ip" = "127.0.0.1; ls -la /root"
      Key: ip
      Value: 127.0.0.1; ls -la /root
```

Figure 27: Wireshark form dissector — ip = "127.0.0.1; ls -la /root" — OS command injection payload captured at packet level

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name
Initial Access	**T1190**	Exploit Public-Facing Application
Execution	**T1059.004**	Command and Scripting Interpreter: Unix Shell
Discovery	**T1083**	File and Directory Discovery
Privilege Escalation	**T1068**	Exploitation for Privilege Escalation
Persistence	**T1505.003**	Server Software Component: Web Shell
Exfiltration	**T1041**	Exfiltration Over C2 Channel

Recommendations & Remediation

The following controls are critical to prevent OS command injection attacks:

- Never pass user-supplied input directly to OS shell functions — refactor the ping utility to use a language-native ICMP library (e.g., Python's subprocess with strict argument list, never shell=True) that does not invoke a shell interpreter.
- Implement strict allowlist input validation on the ip parameter — only permit valid IPv4 or IPv6 address formats using regex validation before any processing.
- If shell execution is absolutely required, use argument arrays (never string concatenation) to pass parameters, preventing metacharacter injection entirely.
- Disable or restrict the diagnostic ping endpoint from public access — place it behind authentication and restrict it to internal/admin users only.
- Deploy a WAF rule to detect and block common command injection delimiters (;, |, &&, `, $ ()) in POST form body parameters.
- Conduct an emergency review of /root directory contents — the ls -la /root payload was sent 7 times and may have successfully returned sensitive files, SSH keys, or configuration data.
- Implement Elastic SIEM alerting for POST requests to diagnostic endpoints containing shell metacharacters (;, |, &&) in form body parameters.
- Perform a full forensic audit of the server to determine whether subsequent commands (cat /etc/shadow, wget, reverse shell) were executed following the initial ls -la /root probe.